

part b draft xxxx

2025-11-03

Loading the Required Packages

```
pacman::p_load(tidyverse, dplyr, readr, here, sf, tmap, corrplot, spdep, janitor, here, pheatmap, caret
```

```
library(MASS)
```

```
##  
## Attaching package: 'MASS'  
  
## The following object is masked from 'package:dplyr':  
##  
##      select
```

```
library(dplyr)
```

Loading the Datasets

- This script loads the Tesco grocery data, MSOA demographic data, and the London shapefile, cleans all their MSOA names and codes, and matches them so every dataset lines up properly.
- It removes unwanted rows, checks which MSOAs matched or didnt, and joins everything into one complete dataset that includes map shapes, food purchases, and demographic indicators. After that, it drops the geometry to create a clean modelling table, removes missing values, and saves both the spatial file and the merged CSV.
- Finally, the script loops through each predictor variable, creates a scatterplot with a fitted regression line against energy density, and saves every plot as a PNG in the figs folder.

```
# 0) Libraries  
library(readr)  
library(dplyr)  
library(stringr)  
library(janitor)  
library(sf)  
library(ggplot2)  
  
# 1) File paths (edit if folders differ)  
path_tesco <- "data/year_msoa_grocery.csv"  
path_atlas <- "data/msoa-data.csv"      # CSV downloaded from MSOA Atlas  
path_shp   <- "data/statistical-gis-boundaries-london/statistical-gis-boundaries-london/ESRI/MSOA_2011_1"
```

```

# 2) Tesco (has MSOA code)
tesco <- readr::read_csv(path_tesco, show_col_types = FALSE) |>
  dplyr::rename(msoa_code = area_id) |>
  dplyr::mutate(msoa_code = toupper(stringr::str_trim(as.character(msoa_code))))

# 3) Atlas (handle encoding; clean names)
atlas <- readr::read_csv(path_atlas, locale = readr::locale(encoding = "UTF-8"), show_col_types = FALSE)
janitor::clean_names()

## New names:
## * `House Prices;Sales;2011` -> `House Prices;Sales;2011...129`
## * `House Prices;Sales;2011` -> `House Prices;Sales;2011...130`

if (!"msoa_name" %in% names(atlas)) stop("Atlas file must contain a column named 'msoa_name'.")

# Remove aggregate rows such as "AVERAGE", "ALL LONDON", etc.
atlas <- atlas |>
  dplyr::filter(!stringr::str_detect(stringr::str_to_upper(coalesce(msoa_name, "")),
    "AVERAGE|TOTAL|ALL LONDON|GREATER LONDON"))

# 4) Shapefile (2011). Normalise to msoa_code/msoa_name
msoas_raw <- sf::st_read(path_shp, quiet = TRUE)

# Try common field names (works for 2011/2021 shapefiles too if you swap files later)
code_col <- intersect(names(msoas_raw), c("msoa_code", "MSOA21CD", "MSOA11CD"))[1]
name_col <- intersect(names(msoas_raw), c("msoa_name", "MSOA21NM", "MSOA11NM"))[1]
if (is.na(code_col) || is.na(name_col)) {
  stop(sprintf("Could not find MSOA code/name fields in the shapefile. Found: %s",
    paste(names(msoas_raw), collapse = ", ")))
}

msoas <- msoas_raw |>
  dplyr::rename(msoa_code = !!code_col, msoa_name = !!name_col) |>
  dplyr::mutate(msoa_code = toupper(stringr::str_trim(as.character(msoa_code))))

# 5) Consistent uppercase names (lookup)
msoas_lookup <- msoas |>
  sf::st_drop_geometry() |>
  dplyr::mutate(
    msoa_name = msoa_name |>
      stringr::str_to_upper() |>
      stringr::str_replace_all("&", "AND") |>
      stringr::str_replace_all("[^A-Z0-9 ]", " ") |>
      stringr::str_squish()
  ) |>
  dplyr::select(msoa_name, msoa_code)

atlas_for_join <- atlas |>
  dplyr::mutate(
    msoa_name = msoa_name |>
      stringr::str_to_upper() |>
      stringr::str_replace_all("&", "AND") |>
      stringr::str_replace_all("[^A-Z0-9 ]", " ") |>

```

```

    stringr::str_squish()
  )

# 6) Join diagnostics
cat("# rows - tesco:", nrow(tesco), " atlas:", nrow(atlas), " msoas:", nrow(msoas), "\n")

## # rows - tesco: 983 atlas: 983 msoas: 983

atlas_with_code <- dplyr::inner_join(atlas_for_join, msoas_lookup, by = "msoa_name")
cat("Matched atlas+MSOA by name:", nrow(atlas_with_code), "rows\n")

## Matched atlas<U+2192>MSOA by name: 983 rows

unmatched_atlas <- dplyr::anti_join(atlas_for_join, msoas_lookup, by = "msoa_name")
cat("Unmatched atlas names:", nrow(unmatched_atlas), "\n")

## Unmatched atlas names: 0

if (nrow(unmatched_atlas) > 0) readr::write_csv(unmatched_atlas, "data/_unmatched_atlas_names.csv")

# 7) Tesco + Atlas by code
tesco_atlas <- dplyr::inner_join(
  tesco,
  dplyr::select(atlas_with_code, msoa_code, dplyr::everything()),
  by = "msoa_code"
)
cat("Rows after Tesco+Atlas join:", nrow(tesco_atlas), "\n")

## Rows after Tesco+Atlas join: 983

# 8) Join with shapefile (sf)
msoa_all <- dplyr::inner_join(msoas, tesco_atlas, by = "msoa_code")
cat("Rows after spatial join (should be MSA count):", nrow(msoa_all), "\n")

## Rows after spatial join (should be MSA count): 983

# 9) Plain data frame for modelling/plots
stats_data <- sf::st_drop_geometry(msoa_all)

# 10) Remove missing values from the stats_data
stats_data_clean <- na.omit(stats_data) # Remove rows with any missing values

# Save clean outputs
readr::write_csv(stats_data, "data/merged_tesco_atlas.csv")
sf::st_write(msoa_all, "data/msoa_all.gpkg", delete_dsn = TRUE, quiet = TRUE)

# 11) Scatterplot loop
predictors <- c(
  "avg_age", "people_per_sq_km", "age_65+", "man_day", "transaction_days",

```

```

    "num_transactions", "population", "male", "female",
    "f_readymade", "f_wine", "f_soft_drinks", "f_fruit_veg", "f_sweets",
    "f_meat_red", "f_dairy", "f_eggs", "f_fish", "f_grains", "f_poultry", "f_water"
  )
  predictors <- intersect(predictors, names(stats_data))

  if (!dir.exists("figs")) dir.create("figs", recursive = TRUE)

  coerce_num <- function(v) { if (is.numeric(v)) v else suppressWarnings(as.numeric(v)) }

  for (x in predictors) {
    tmp <- stats_data[, c(x, "energy_density")]
    tmp[[x]] <- coerce_num(tmp[[x]])
    tmp[["energy_density"]] <- coerce_num(tmp[["energy_density"]])

    p <- ggplot2::ggplot(tmp, ggplot2::aes(x = .data[[x]], y = .data[["energy_density"]])) +
      ggplot2::geom_point(alpha = 0.6, na.rm = TRUE) +
      ggplot2::geom_smooth(method = "lm", se = TRUE) +
      ggplot2::labs(
        title = paste("Energy density vs", x),
        x = x,
        y = "Energy density (kcal/g)"
      ) +
      ggplot2::theme_minimal()

    print(p)
    ggplot2::ggsave(
      file.path("figs", paste0("scatter_", gsub("[^A-Za-z0-9_]", "_", x), ".png")),
      p, width = 7, height = 5, dpi = 300
    )
  }
}

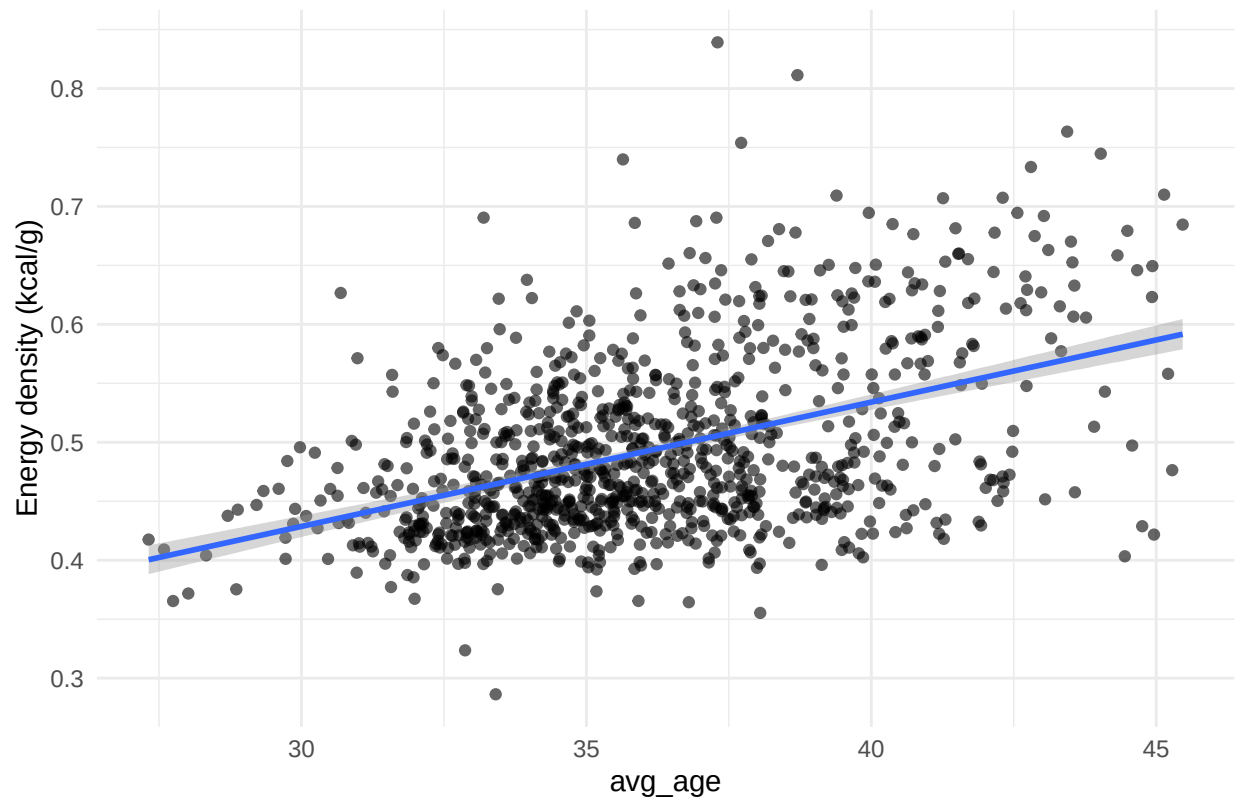
```

```

## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'

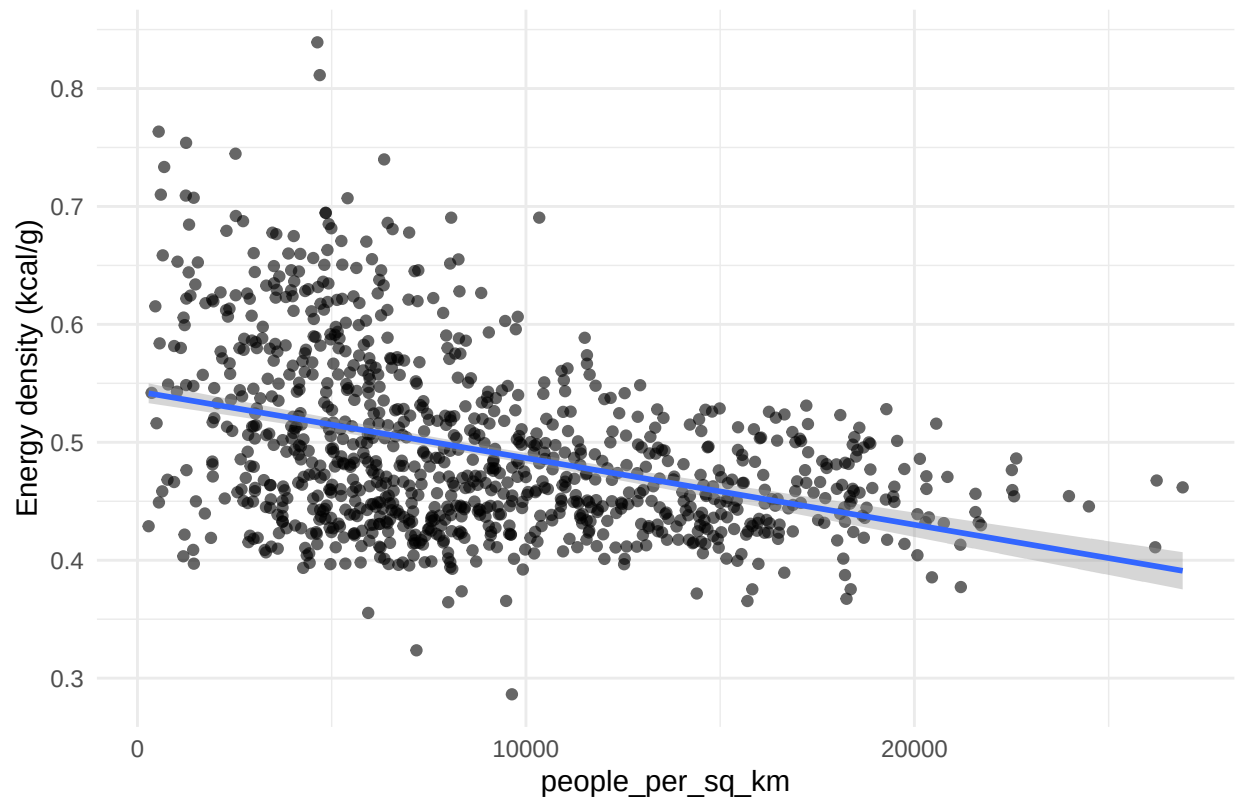
```


Energy density vs avg_age



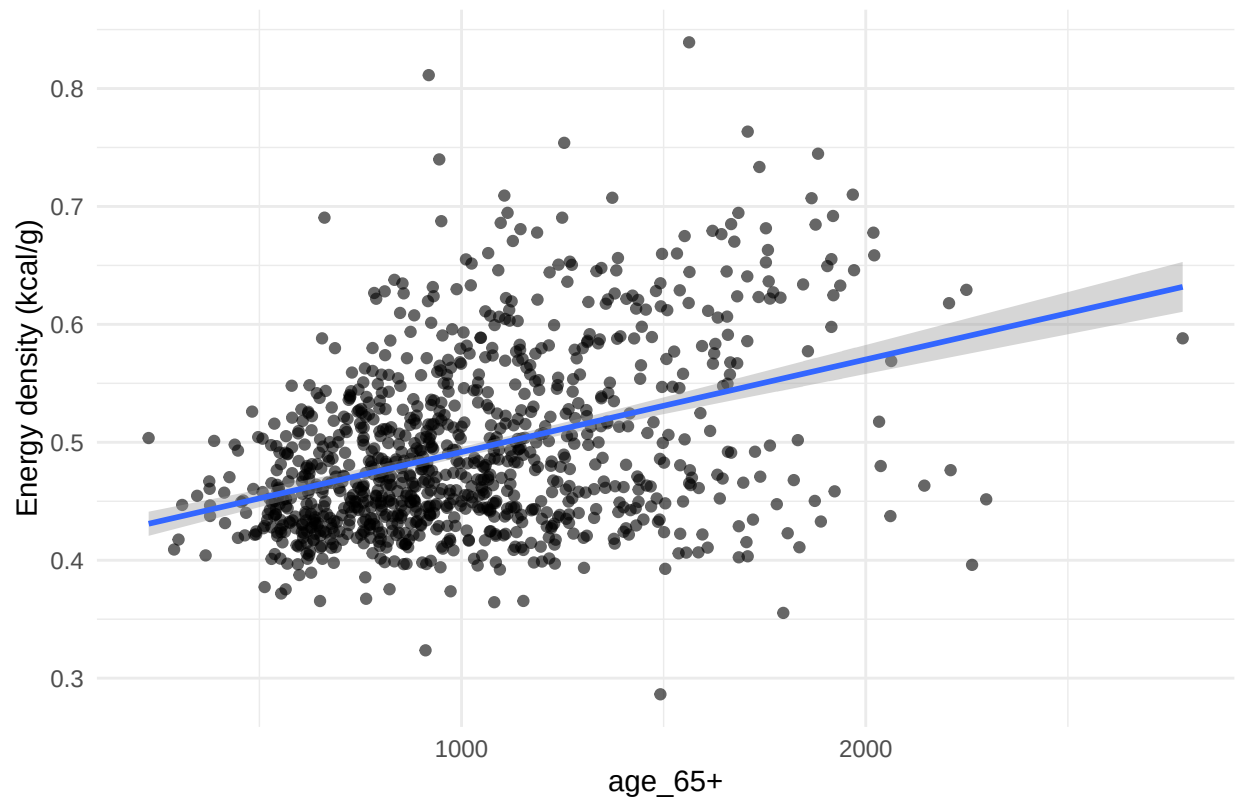
```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```

Energy density vs people_per_sq_km

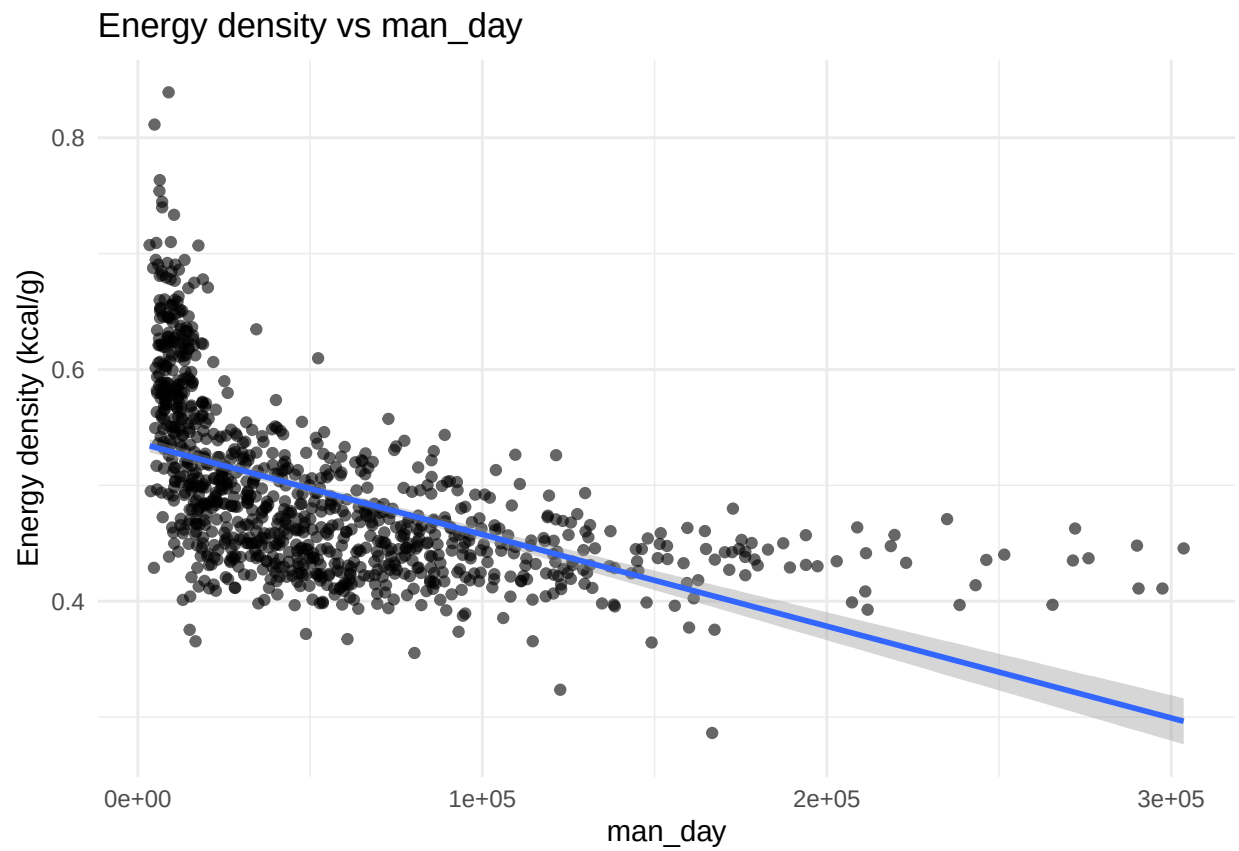


```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```

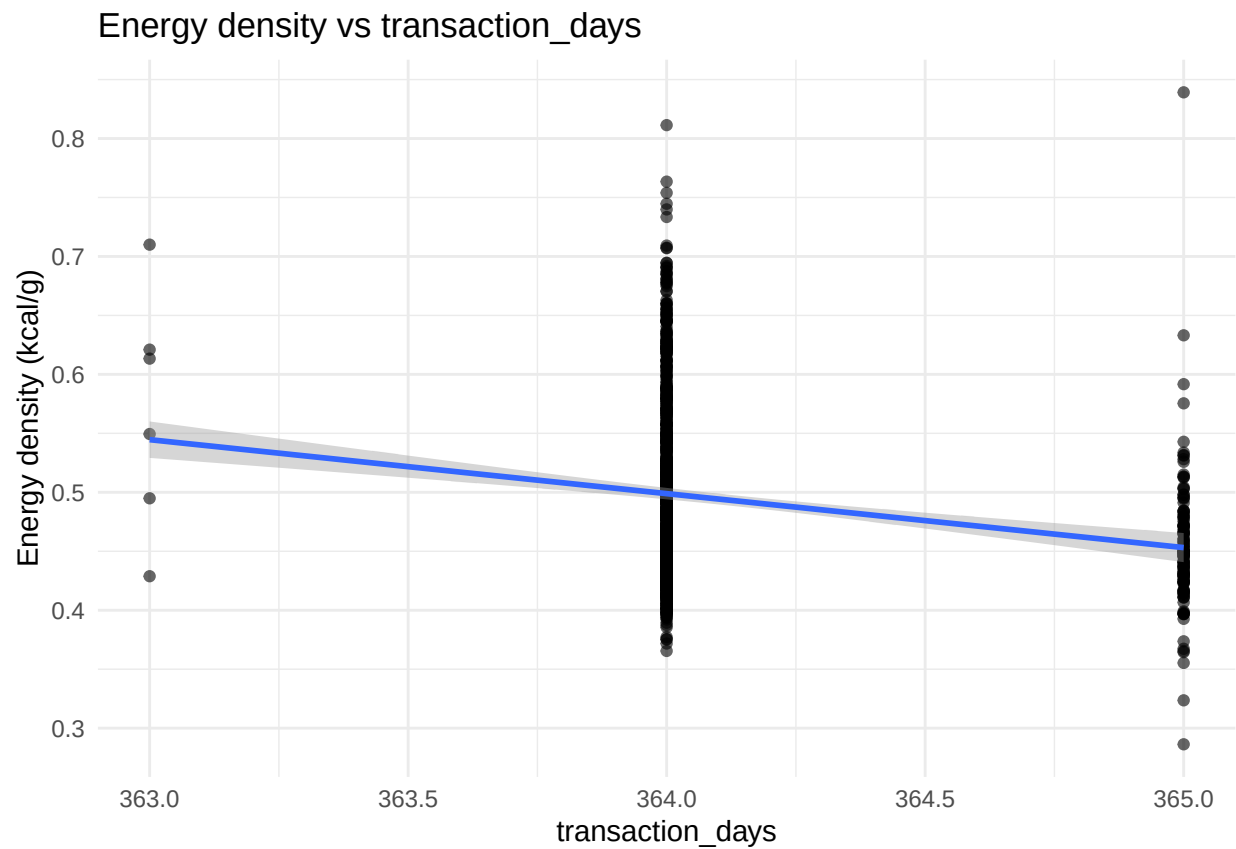
Energy density vs age_65+



```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```

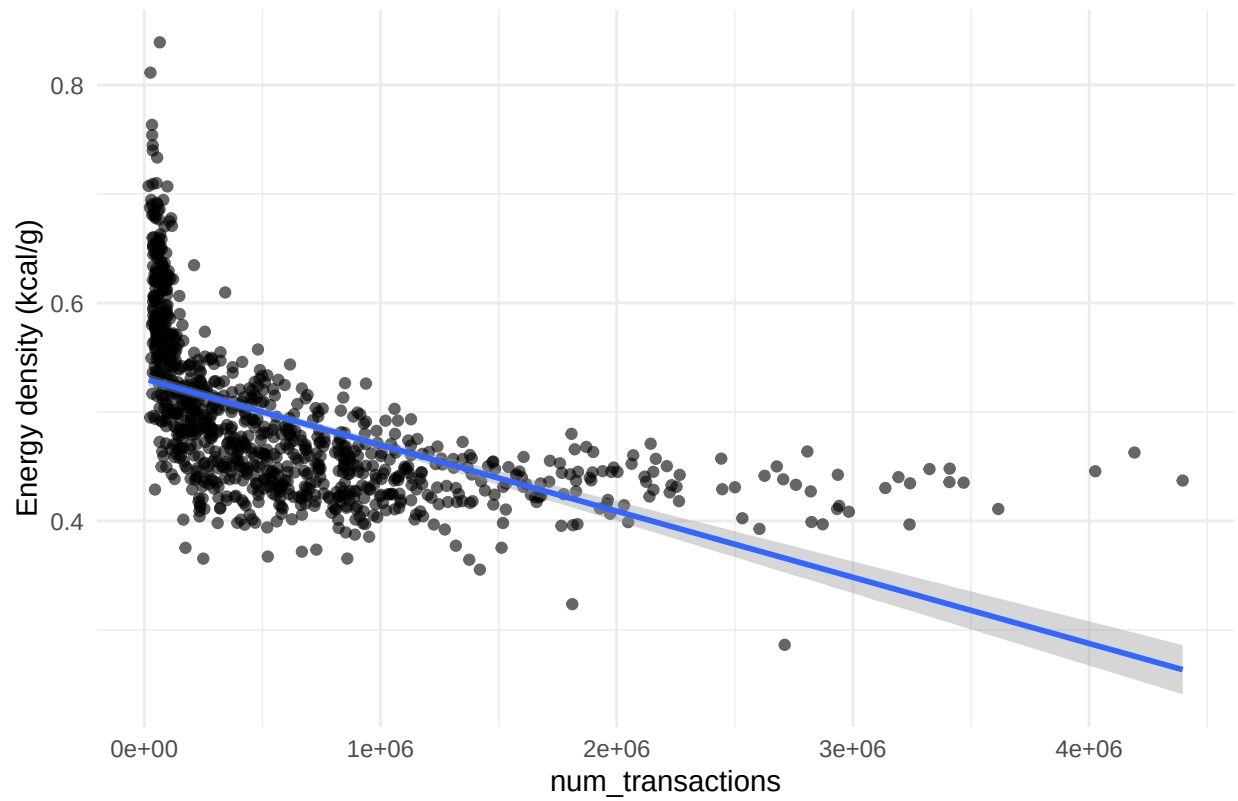


```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```



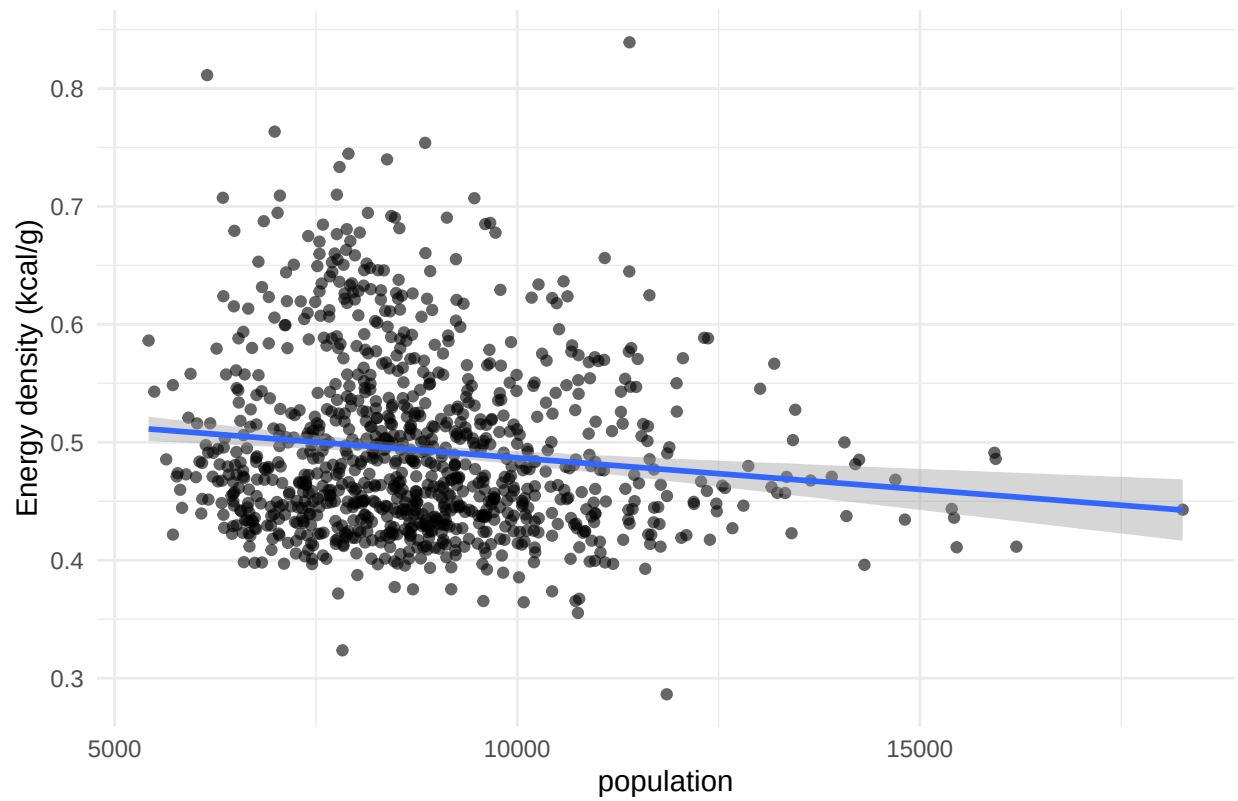
```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```

Energy density vs num_transactions



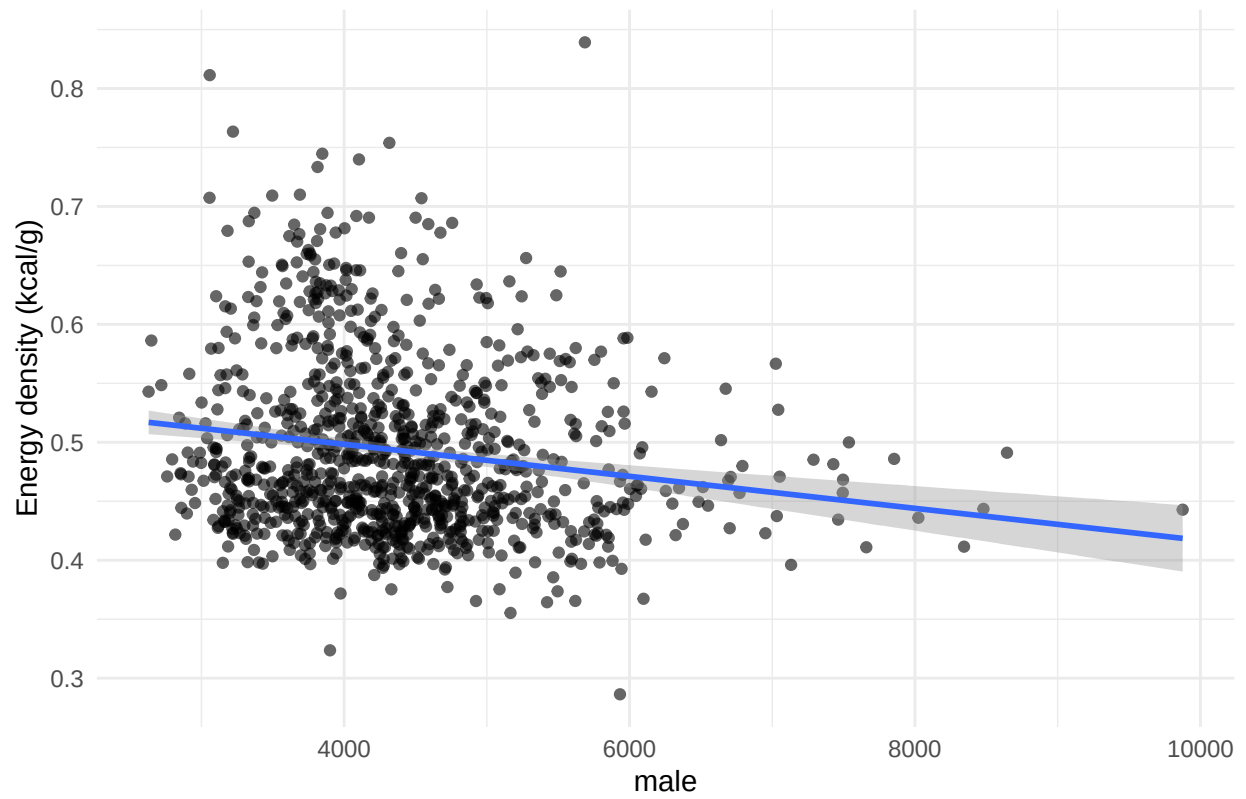
```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```

Energy density vs population



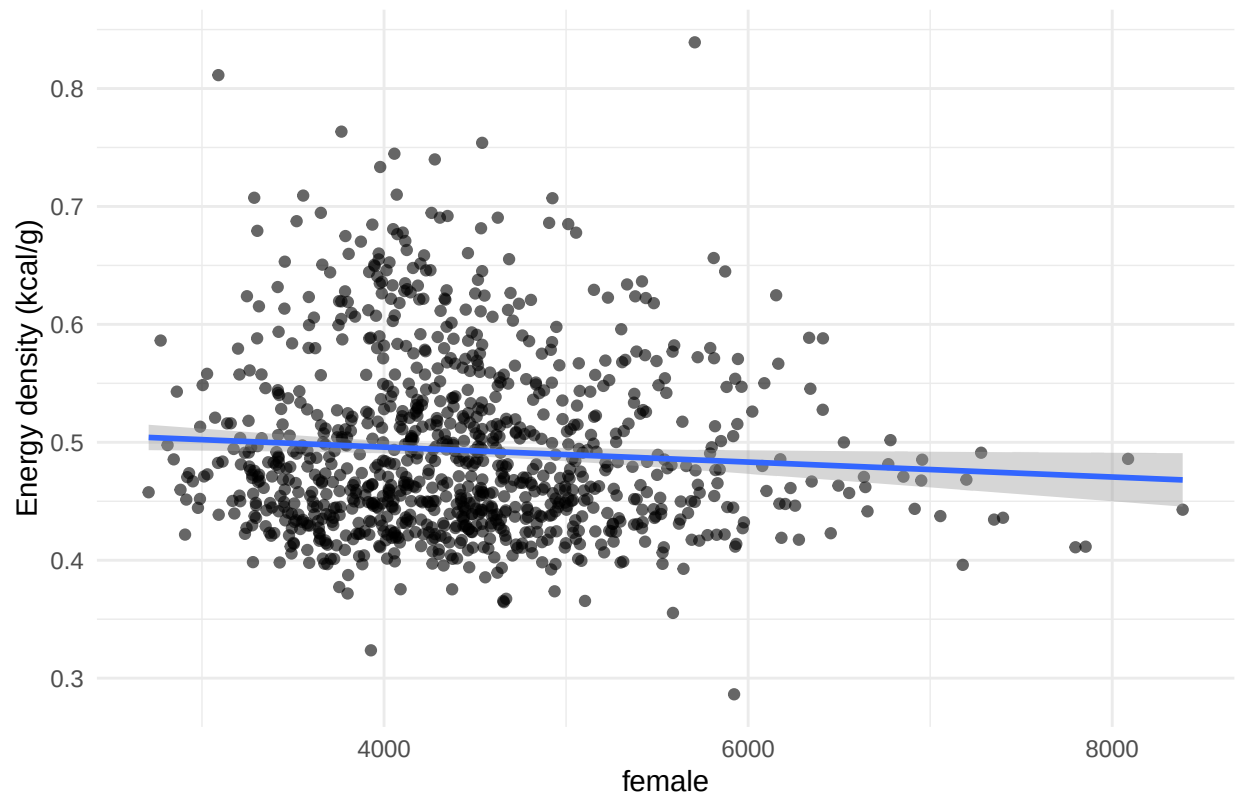
```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```

Energy density vs male

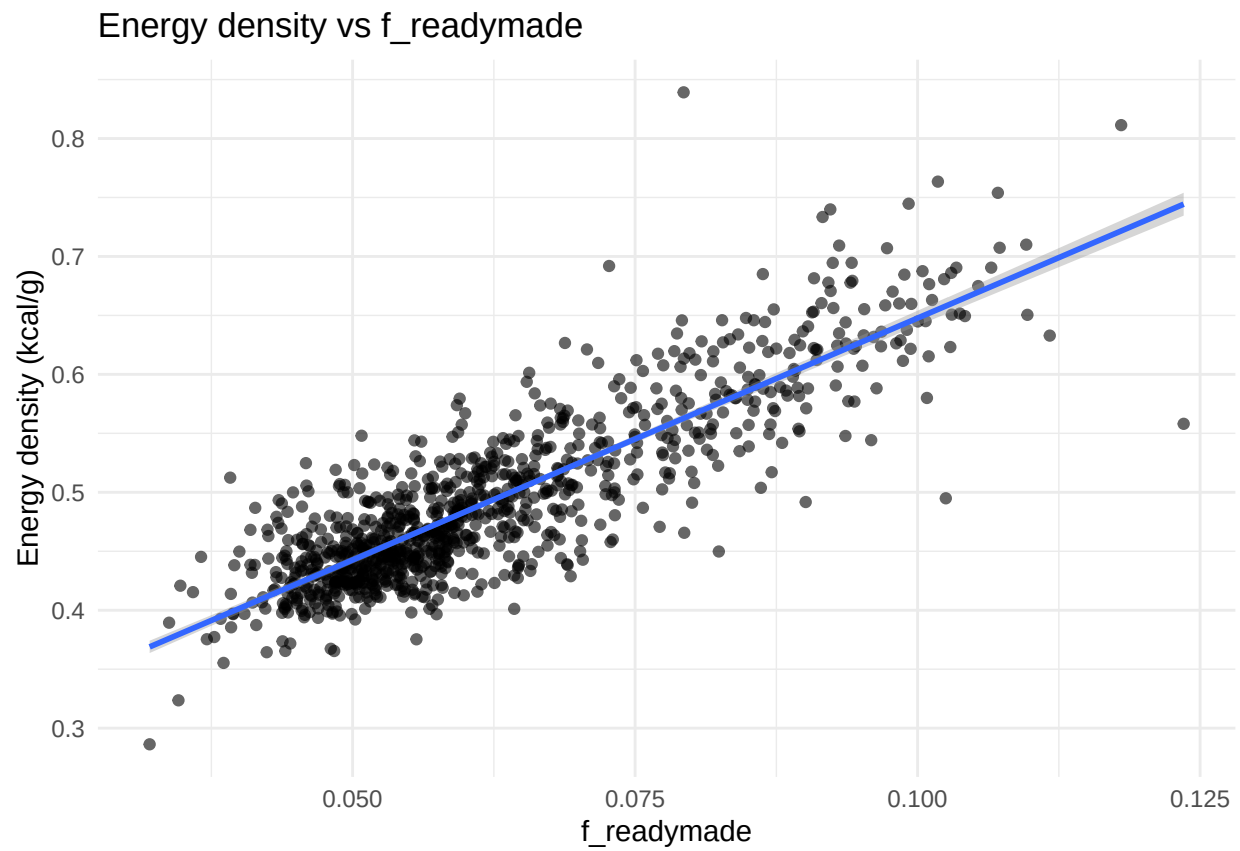


```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```

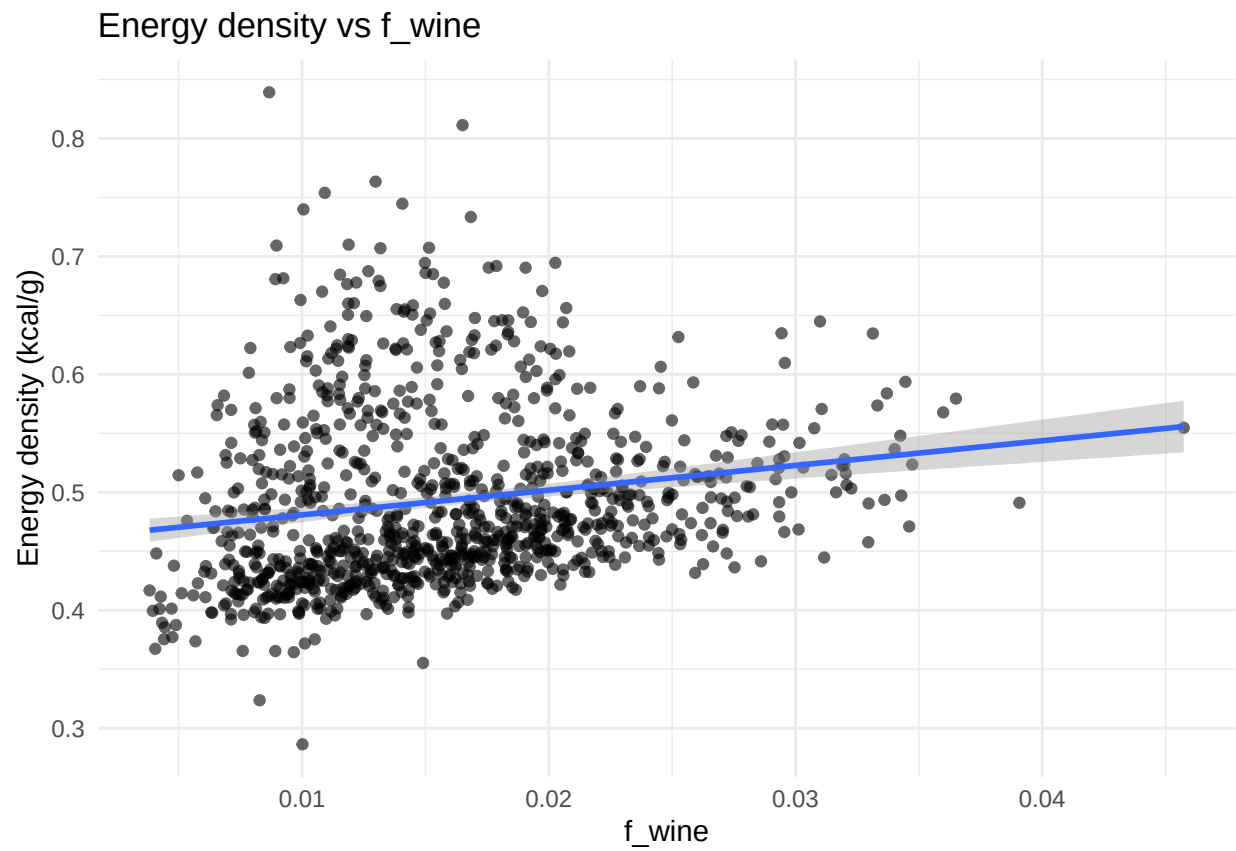

Energy density vs female



```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```

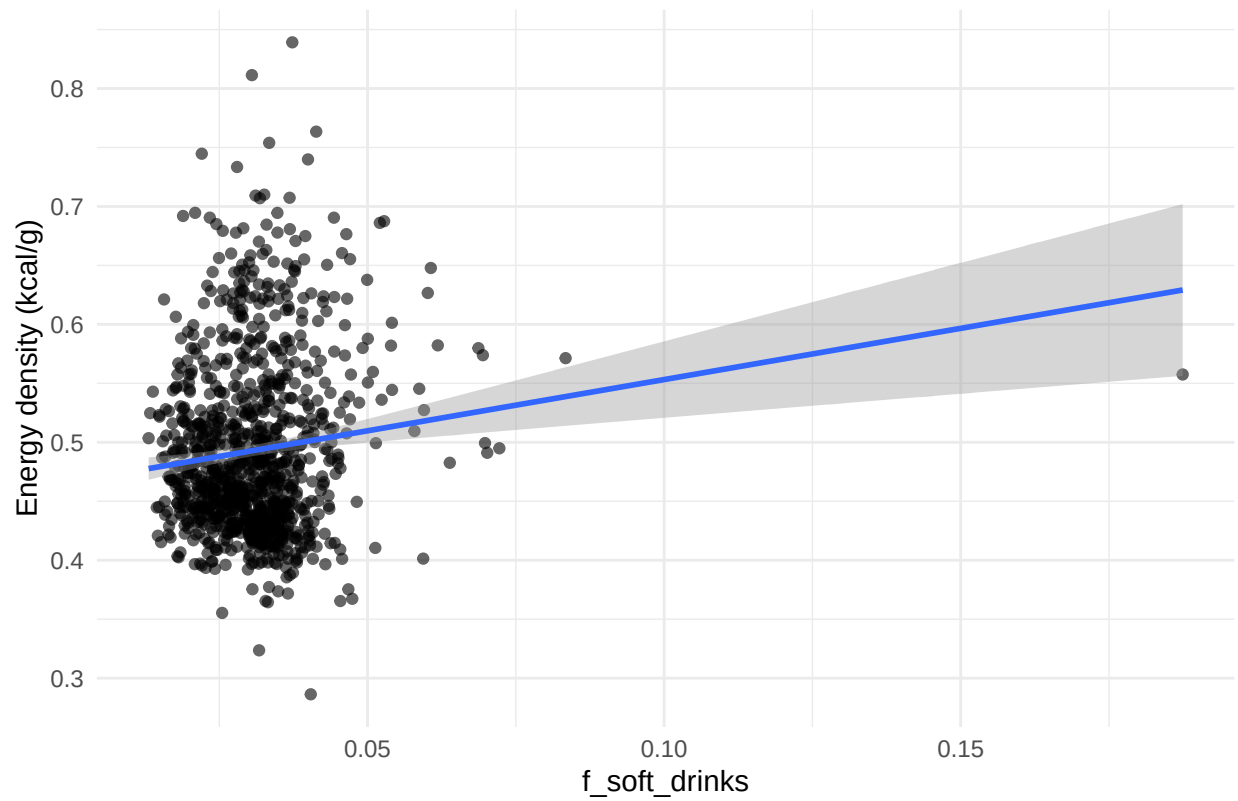


```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```

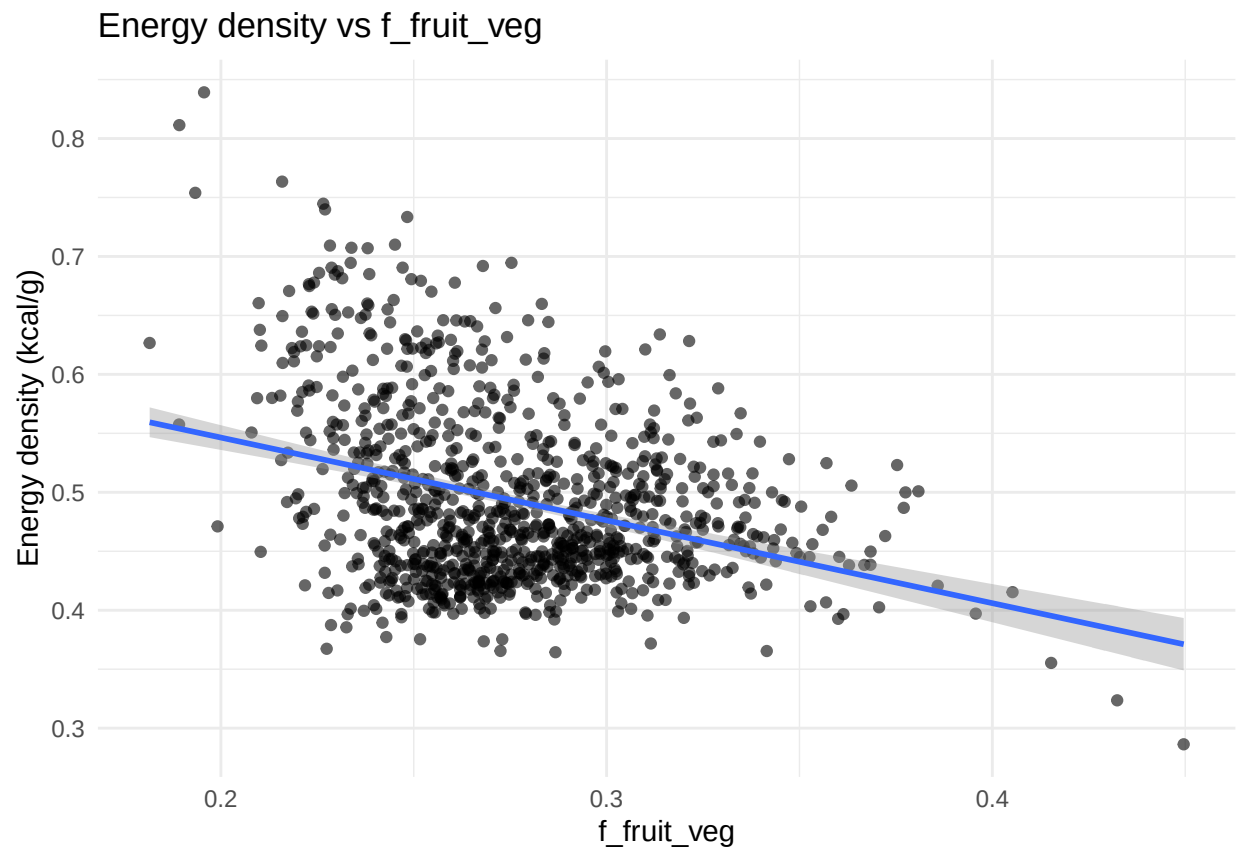


```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```

Energy density vs f_soft_drinks

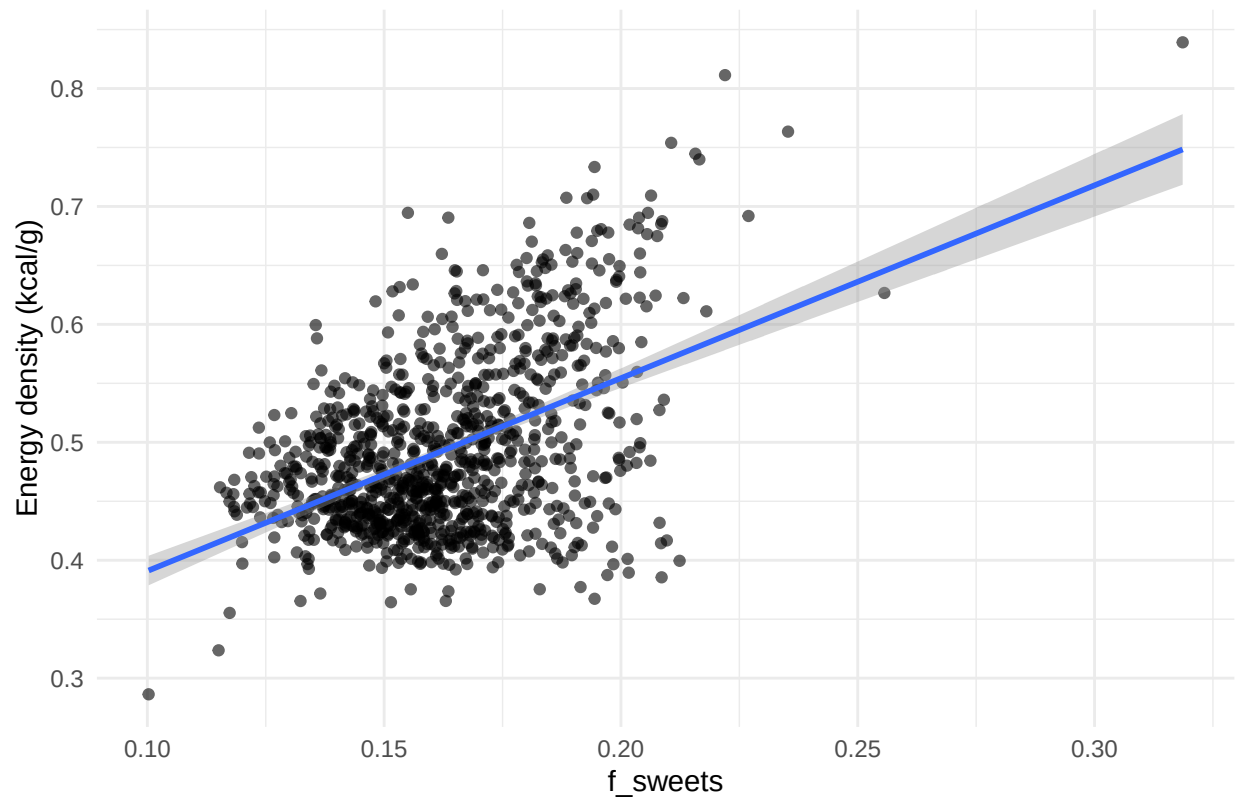


```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```



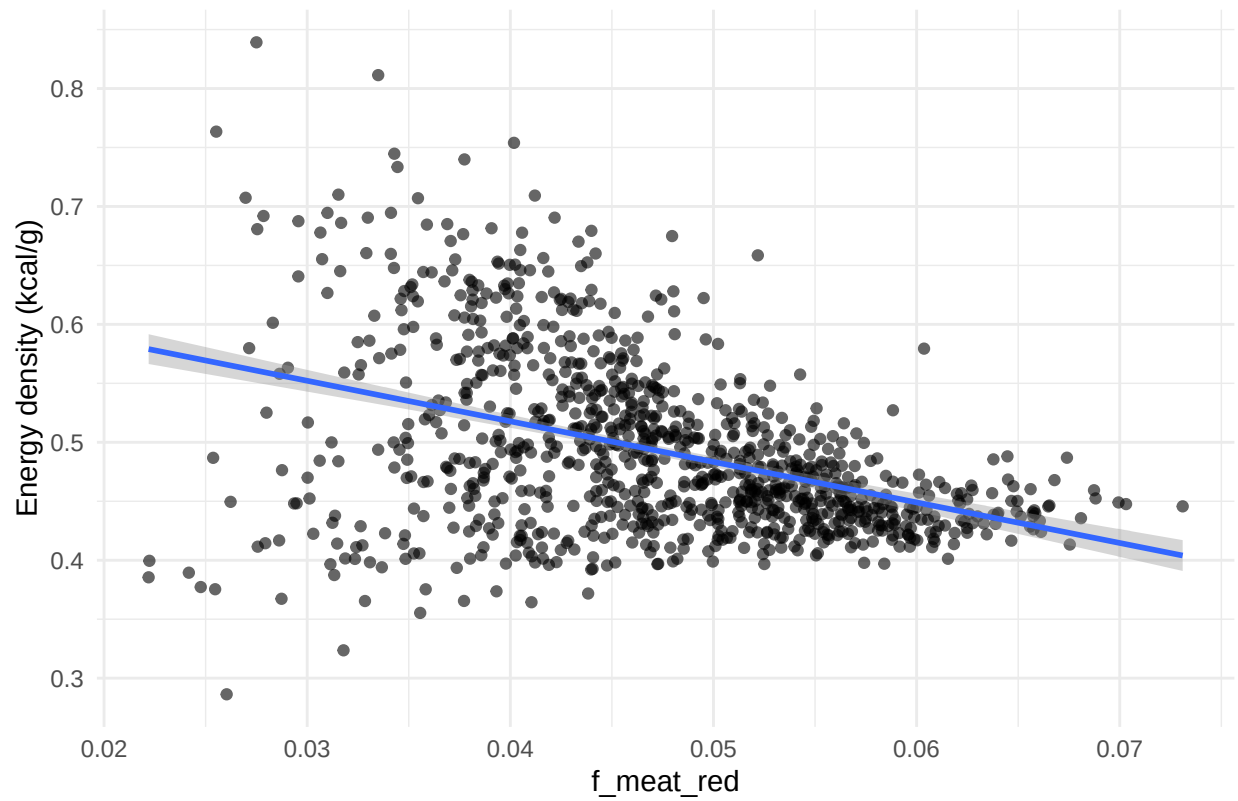
```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```

Energy density vs f_sweets

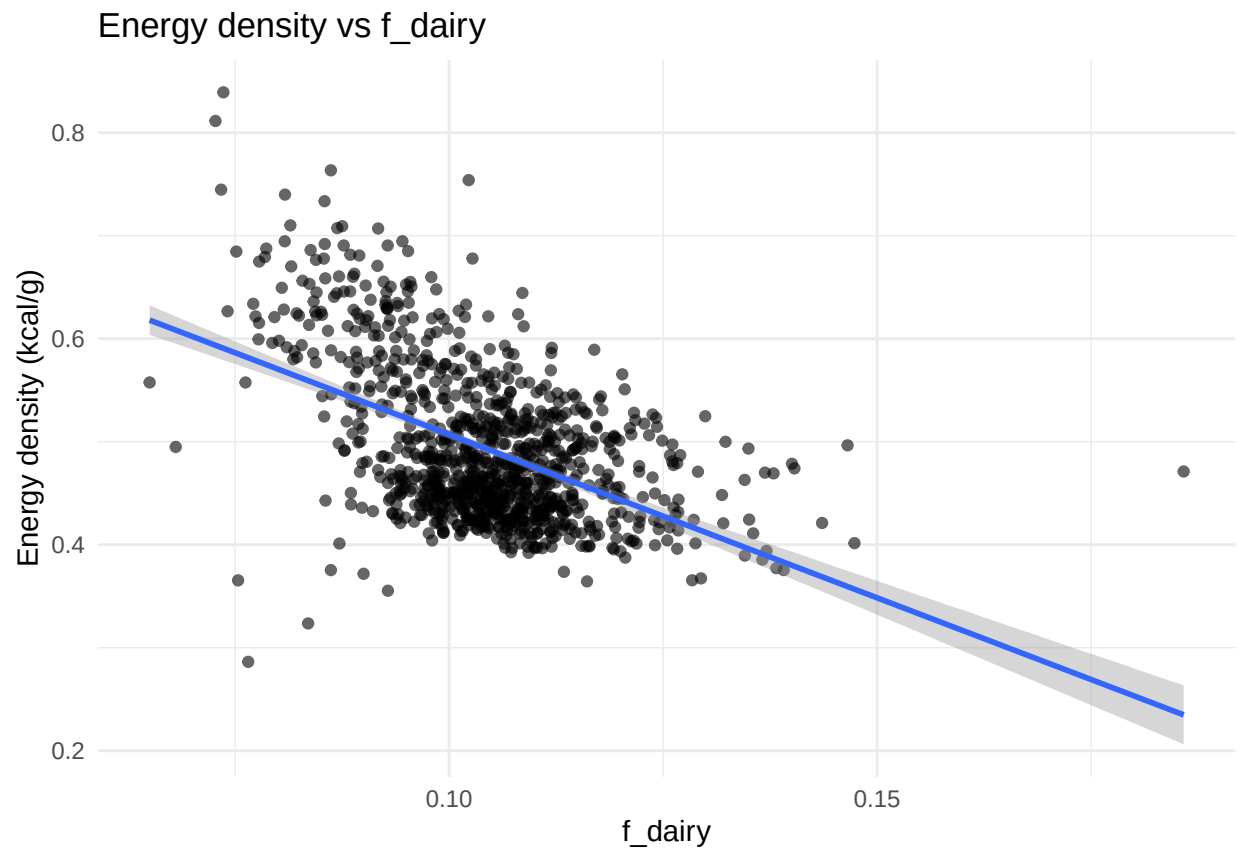


```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```

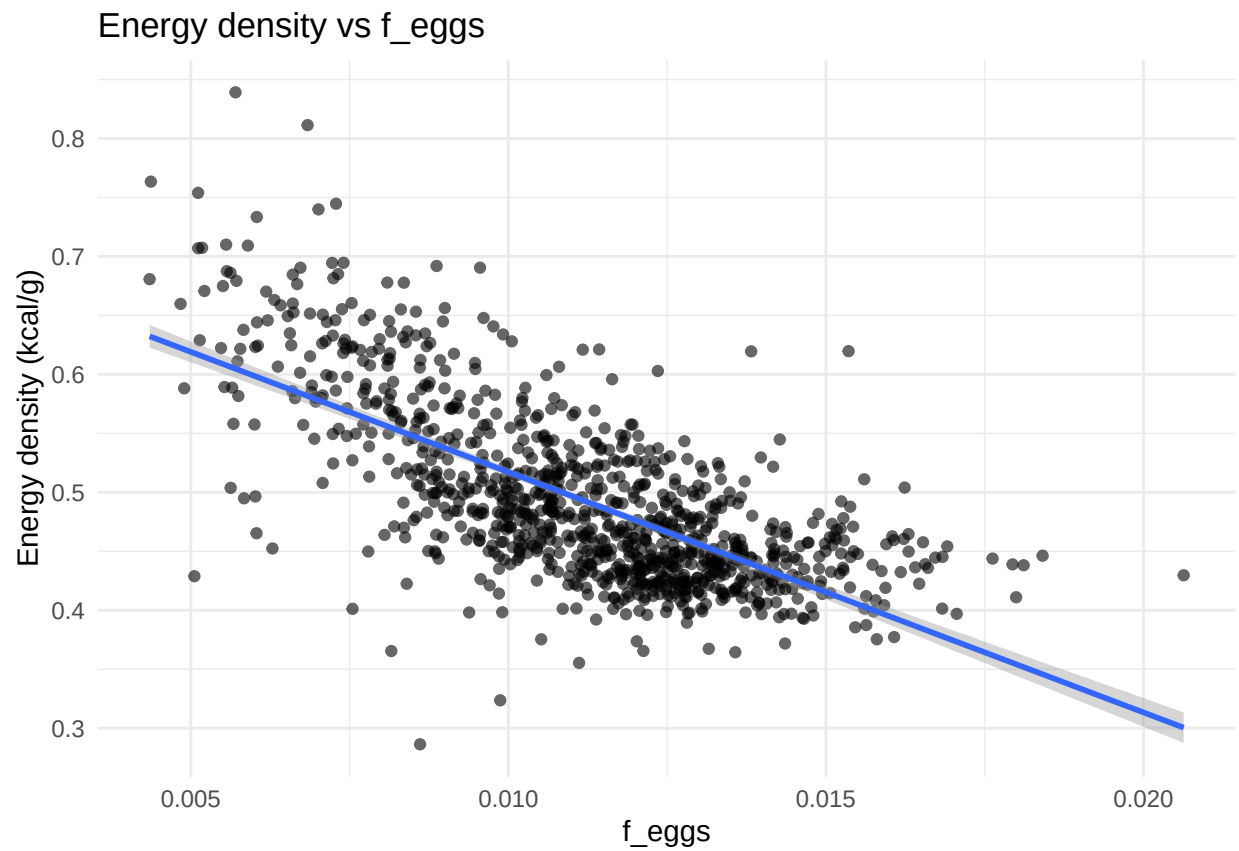
Energy density vs f_meat_red



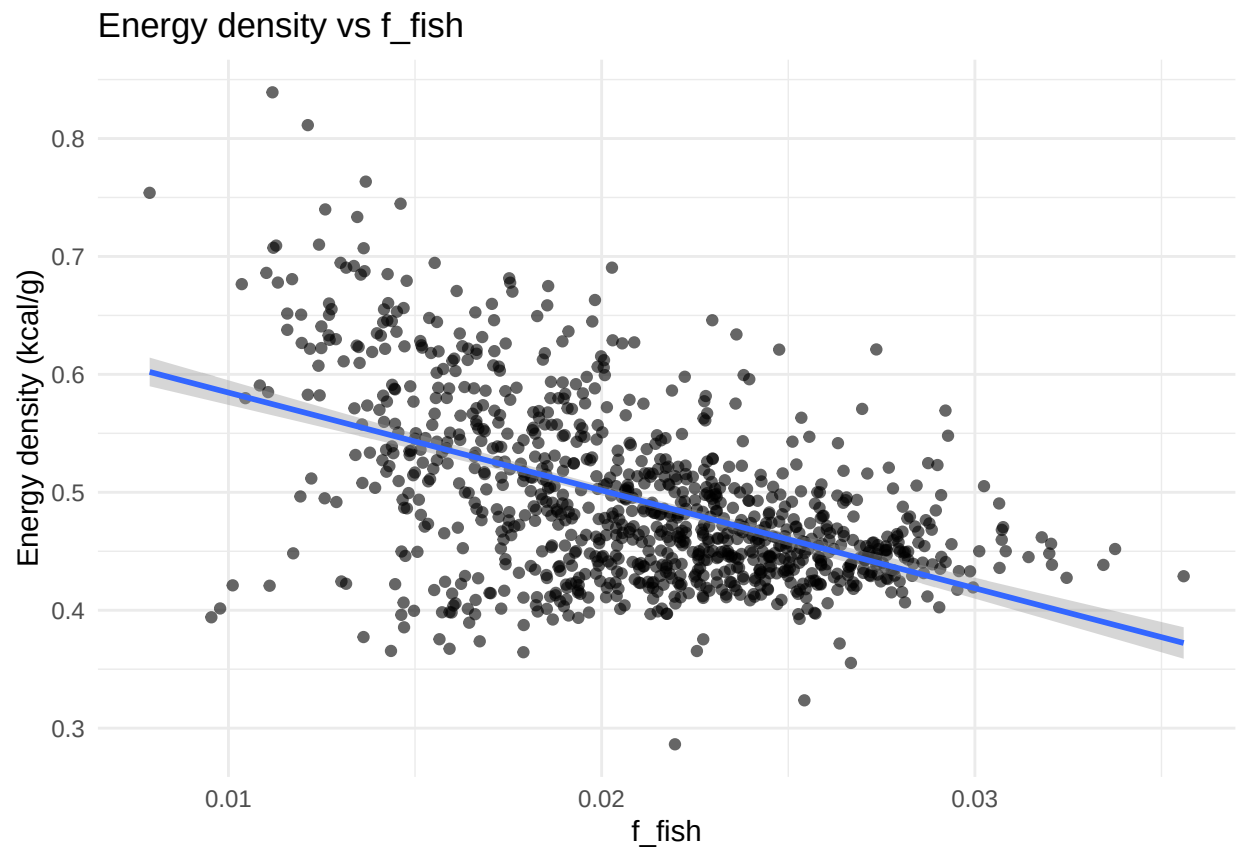
```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```



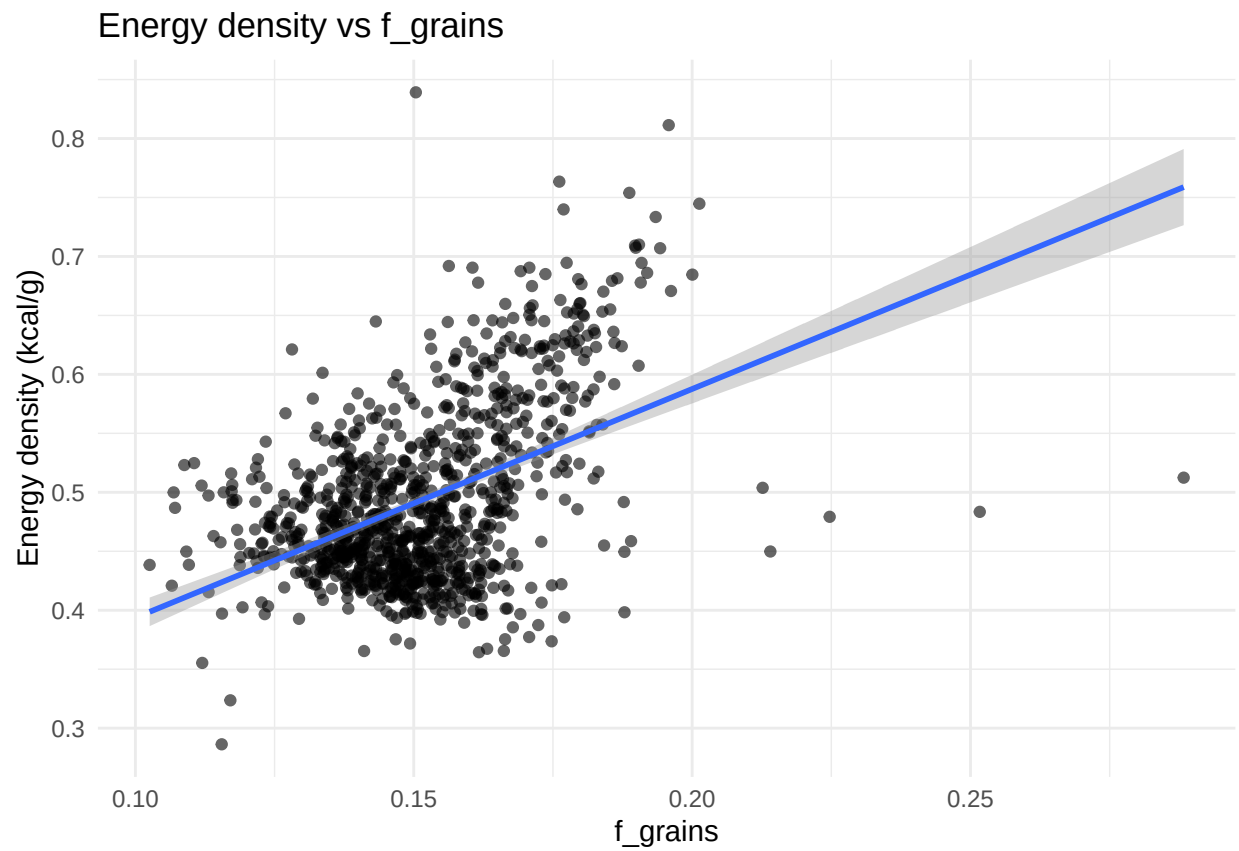
```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```

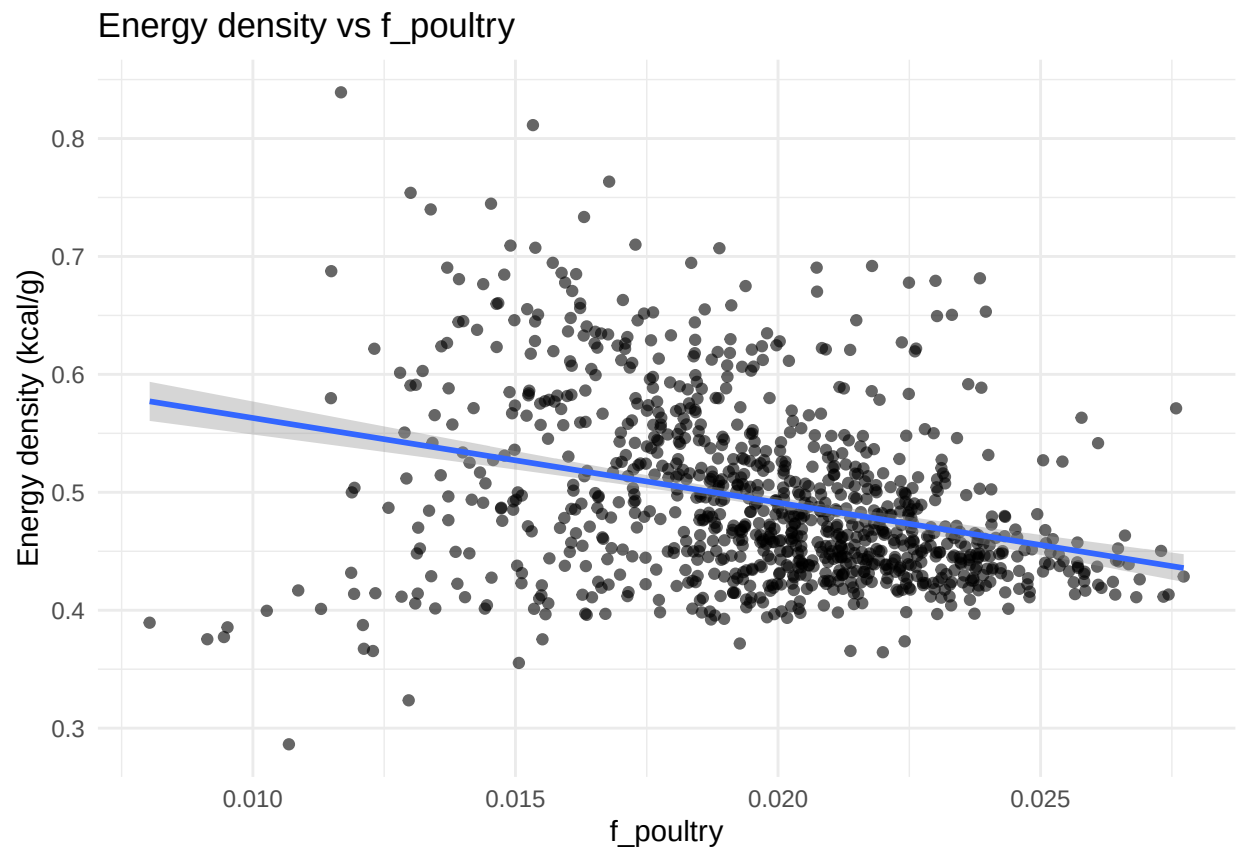
```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```



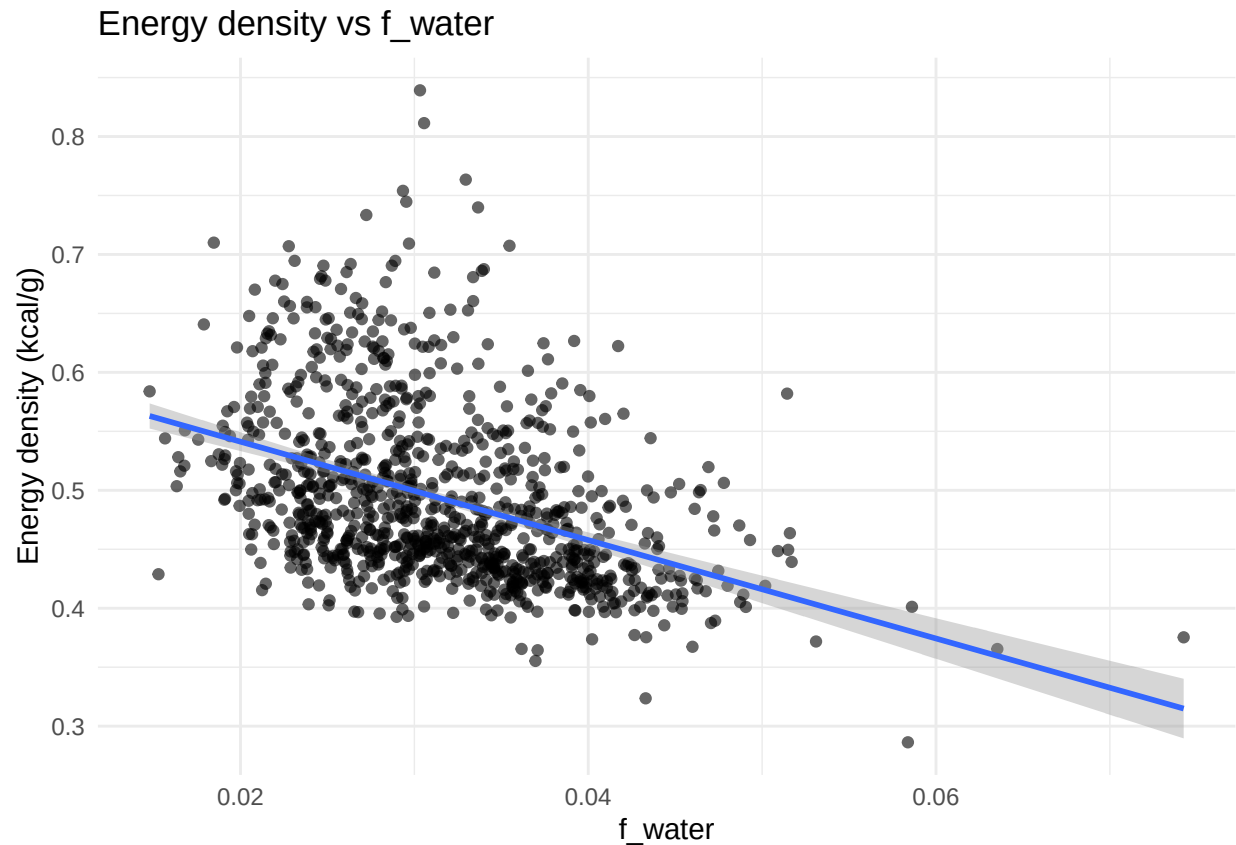
```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```



```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```



```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```



Map Of Energy Density before modeling.

```
# ---- ENERGY DENSITY MAP (Before Modelling) ----
library(tmap)
library(dplyr)

# Ensuring correct column name for code
names(msoa_all)[grepl("msoa_code", names(msoa_all), ignore.case = TRUE)] <- "msoa_code"

# Checking that energy_density exists
if(!"energy_density" %in% names(msoa_all)) {
  stop("energy_density column not found in msoa_all dataset.")
}

# Plot map
tmap_mode("plot")

## i tmap modes "plot" - "view"
## i toggle with `tmap::ttm()`

energy_map <- tm_shape(msoa_all) +
  tm_polygons("energy_density",
```

```

        style = "quantile",
        n = 5,
        palette = "YlOrRd",
        title = "Energy Density (kcal/g)") +
tm_layout(main.title = "Regional Distribution of Energy Density (Pre-Model)",
          legend.outside = TRUE)

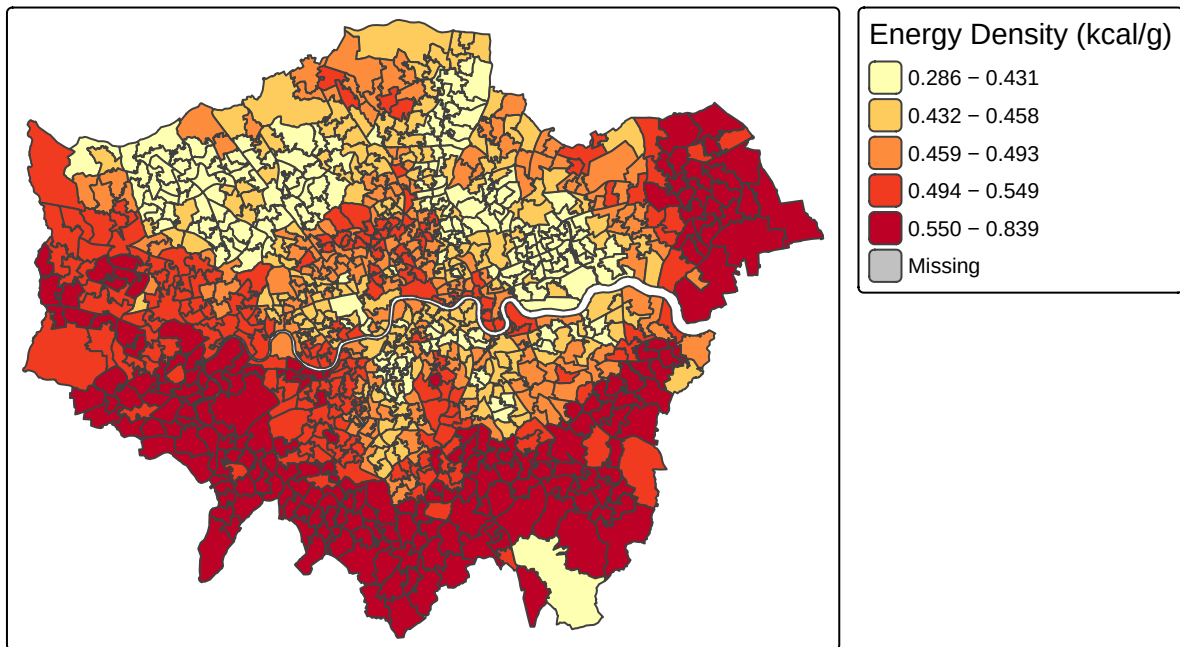
##
## -- tmap v3 code detected -----
## [v3->v4] `tm_polygons()` : instead of `style = "quantile"`, use fill.scale =
## `tm_scale_intervals()`.
## i Migrate the argument(s) 'style', 'n', 'palette' (rename to 'values') to
## `tm_scale_intervals(<HERE>)'[v3->v4] `tm_polygons()` : migrate the argument(s) related to the legend
## visual variable `fill` namely 'title' to 'fill.legend = tm_legend(<HERE>)'[v3->v4] `tm_layout()` : use

# Displaying map
energy_map

## [cols4all] color palettes: use palettes from the R package cols4all. Run
## `cols4all::c4a_gui()` to explore them. The old palette name "YlOrRd" is named
## "brewer.yl_or_rd"
## Multiple palettes called "yl_or_rd" found: "brewer.yl_or_rd", "matplotlib.yl_or_rd". The first one,
##
## [plot mode] fit legend/component: Some legend items or map components do not
## fit well, and are therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.

```

Regional Distribution of Energy Density (Pre-Model)



```
# Saving high-quality PNG
if (!dir.exists("Figures_final")) dir.create("Figures_final", recursive = TRUE)
tmap::tmap_save(energy_map, "Figures_final/energy_density_pre_model.png",
               width = 2200, height = 1600, dpi = 220)

## [cols4all] color palettes: use palettes from the R package cols4all. Run
## `cols4all::c4a_gui()` to explore them. The old palette name "YlOrRd" is named
## "brewer.yl_or_rd"
## Multiple palettes called "yl_or_rd" found: "brewer.yl_or_rd", "matplotlib.yl_or_rd". The first one,
##
## Map saved to U:\Research project\Figures_final\energy_density_pre_model.png
##
## Resolution: 2200 by 1600 pixels
##
## Size: 10 by 7.272727 inches (220 dpi)
```

Checking the Dimensions of Dataset

```
dim(stats_data)      # rows, columns
```

```
## [1] 983 420
```

```
ncol(stats_data)      # just the number of variables
```

```
## [1] 420
```

```
names(stats_data)[1:20] # peek at the first 20 column names
```

```
## [1] "msoa_code"      "msoa_name.x"    "LAD11CD"        "LAD11NM"
## [5] "RGN11CD"        "RGN11NM"        "USUALRES"       "HHOLDRES"
## [9] "COMESTRES"      "POPDEN"         "HHOLDS"         "AVHHOLDSZ"
## [13] "weight"         "weight_perc2.5" "weight_perc25"  "weight_perc50"
## [17] "weight_perc75"  "weight_perc97.5" "weight_std"     "weight_ci95"
```

Applying the Stepwise Regression to the Variables

```
# =====
# Stepwise Regression on Variables
# =====
```

```
library(MASS) # For stepAIC
```

```
# 0) Safety checks
```

```
stopifnot(exists("stats_data"))
stopifnot("energy_density" %in% names(stats_data))
```

```
# 1) List of variables
```

```
selected_variables <- c(
  "household_composition_2011_percentages_couple_household_without_dependent_children",
  "household_composition_2011_percentages_lone_parent_household",
  "household_composition_2011_percentages_one_person_household",
  "household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main_language",
  "qualifications_2011_census_no_qualifications",
  "qualifications_2011_census_schoolchildren_and_full_time_students_age_18_and_over",
  "adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependent_children",
  "household_income_estimates_2011_12_total_median_annual_household_income_u_fffd",
  "income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households",
  "obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12",
  "obesity_percentage_of_the_population_aged_16_with_a_bmi_of_30_modelled_estimate_2006_2008",
  "people_per_sq_km",
  "male"
)
```

```
# 2) Keeping only the variables
```

```
model_data <- stats_data[, c("energy_density", selected_variables), drop = FALSE]
```

```
# 3) Cleaning data (remove NAs for the selected variables)
```

```
model_data_clean <- na.omit(model_data)
```

```
# 4) Running stepwise regression
```

```
# Using both directions: forward and backward selection
```



```
stepwise_model <- stepAIC(lm(energy_density ~ ., data = model_data_clean), direction = "both", trace = 1)

# 5) Getting the selected variables after stepwise regression
selected_vars_stepwise <- attr(terms(stepwise_model), "term.labels")
cat("Final selected variables after stepwise regression:\n")
```

```
## Final selected variables after stepwise regression:
```

```
print(selected_vars_stepwise)
```

```
## [1] "household_composition_2011_percentages_couple_household_without_dependent_children"
## [2] "household_composition_2011_percentages_lone_parent_household"
## [3] "household_composition_2011_percentages_one_person_household"
## [4] "household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main_language"
## [5] "adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependent_children"
## [6] "household_income_estimates_2011_12_total_median_annual_household_income_u_fffd"
## [7] "income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households"
## [8] "obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12"
```

```
# 6) Getting the model summary
summary(stepwise_model)
```

```
##
## Call:
## lm(formula = energy_density ~ household_composition_2011_percentages_couple_household_without_dependent_children +
##     household_composition_2011_percentages_lone_parent_household +
##     household_composition_2011_percentages_one_person_household +
##     household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main_language +
##     adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependent_children +
##     household_income_estimates_2011_12_total_median_annual_household_income_u_fffd +
##     income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households +
##     obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12,
##     data = model_data_clean)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.19768 -0.03807 -0.00382  0.03509  0.32728
##
## Coefficients:
##
## (Intercept)
## household_composition_2011_percentages_couple_household_without_dependent_children
## household_composition_2011_percentages_lone_parent_household
## household_composition_2011_percentages_one_person_household
## household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main_language
## adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependent_children
## household_income_estimates_2011_12_total_median_annual_household_income_u_fffd
## income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households
## obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12
##
## (Intercept)
```

```
## household_composition_2011_percentages_couple_household_without_dependent_children
## household_composition_2011_percentages_lone_parent_household
## household_composition_2011_percentages_one_person_household
## household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main_language
## adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependent_children
## household_income_estimates_2011_12_total_median_annual_household_income_u_fffd
## income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households
## obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12
##
## (Intercept)
## household_composition_2011_percentages_couple_household_without_dependent_children
## household_composition_2011_percentages_lone_parent_household
## household_composition_2011_percentages_one_person_household
## household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main_language
## adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependent_children
## household_income_estimates_2011_12_total_median_annual_household_income_u_fffd
## income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households
## obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12
##
## (Intercept)
## household_composition_2011_percentages_couple_household_without_dependent_children
## household_composition_2011_percentages_lone_parent_household
## household_composition_2011_percentages_one_person_household
## household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main_language
## adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependent_children
## household_income_estimates_2011_12_total_median_annual_household_income_u_fffd
## income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households
## obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12
##
## (Intercept)
## household_composition_2011_percentages_couple_household_without_dependent_children
## household_composition_2011_percentages_lone_parent_household
## household_composition_2011_percentages_one_person_household
## household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main_language
## adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependent_children
## household_income_estimates_2011_12_total_median_annual_household_income_u_fffd
## income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households
## obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.05723 on 891 degrees of freedom
## Multiple R-squared:  0.4455, Adjusted R-squared:  0.4405
## F-statistic: 89.49 on 8 and 891 DF, p-value: < 2.2e-16
```

```
# 7) Saving the results
if (!dir.exists("tables")) dir.create("tables", recursive = TRUE)
if (!dir.exists("figs"))   dir.create("figs", recursive = TRUE)

# Saving the coefficients of the final model
coef_table <- broom::tidy(stepwise_model)
readr::write_csv(coef_table, "tables/stepwise_model_coefficients.csv")

# Plot the coefficients of the selected variables (final model)
```

```
p_coef <- ggplot(coef_table, aes(x = reorder(term, estimate), y = estimate)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  coord_flip() +
  labs(title = "Stepwise Model: Standardised Coefficients",
       x = "Variables", y = "Standardised Coefficient") +
  theme_minimal(base_size = 12)

# Save the plot
ggsave("figs/stepwise_coefficients.png", p_coef, width = 8, height = 6, dpi = 300)

# Print final selected variables and model summary
cat("\nSelected variables:\n")
```

```
##
## Selected variables:
```

```
print(selected_vars_stepwise)
```

```
## [1] "household_composition_2011_percentages_couple_household_without_dependent_children"
## [2] "household_composition_2011_percentages_lone_parent_household"
## [3] "household_composition_2011_percentages_one_person_household"
## [4] "household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_ma
## [5] "adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_depend
## [6] "household_income_estimates_2011_12_total_median_annual_household_income_u_fffd"
## [7] "income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households"
## [8] "obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12"
```

```
cat("\nModel Summary:\n")
```

```
##
## Model Summary:
```

```
summary(stepwise_model)
```

```
##
## Call:
## lm(formula = energy_density ~ household_composition_2011_percentages_couple_household_without_depend
##     household_composition_2011_percentages_lone_parent_household +
##     household_composition_2011_percentages_one_person_household +
##     household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_mai
##     adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_depend
##     household_income_estimates_2011_12_total_median_annual_household_income_u_fffd +
##     income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households +
##     obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12,
##     data = model_data_clean)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.19768 -0.03807 -0.00382  0.03509  0.32728
##
## Coefficients:
```

```

##
## (Intercept)
## household_composition_2011_percentages_couple_household_without_dependent_children
## household_composition_2011_percentages_lone_parent_household
## household_composition_2011_percentages_one_person_household
## household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main_la
## adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependent_ch
## household_income_estimates_2011_12_total_median_annual_household_income_u_fffd
## income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households
## obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12
##
## (Intercept)
## household_composition_2011_percentages_couple_household_without_dependent_children
## household_composition_2011_percentages_lone_parent_household
## household_composition_2011_percentages_one_person_household
## household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main_la
## adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependent_ch
## household_income_estimates_2011_12_total_median_annual_household_income_u_fffd
## income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households
## obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12
##
## (Intercept)
## household_composition_2011_percentages_couple_household_without_dependent_children
## household_composition_2011_percentages_lone_parent_household
## household_composition_2011_percentages_one_person_household
## household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main_la
## adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependent_ch
## household_income_estimates_2011_12_total_median_annual_household_income_u_fffd
## income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households
## obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12
##
## (Intercept)
## household_composition_2011_percentages_couple_household_without_dependent_children
## household_composition_2011_percentages_lone_parent_household
## household_composition_2011_percentages_one_person_household
## household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main_la
## adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependent_ch
## household_income_estimates_2011_12_total_median_annual_household_income_u_fffd
## income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households
## obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12
##
## (Intercept)
## household_composition_2011_percentages_couple_household_without_dependent_children
## household_composition_2011_percentages_lone_parent_household
## household_composition_2011_percentages_one_person_household
## household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main_la
## adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependent_ch
## household_income_estimates_2011_12_total_median_annual_household_income_u_fffd
## income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households
## obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.05723 on 891 degrees of freedom

```

```
## Multiple R-squared:  0.4455, Adjusted R-squared:  0.4405
## F-statistic: 89.49 on 8 and 891 DF,  p-value: < 2.2e-16
```

Interpretation of the Stepwise Regression Output

The regression output provides a comprehensive overview of the relationship between the selected variables and energy density (in kcal/g). Lets break down the key results:

Model Summary:

Multiple R-squared: 0.4455

This means that 44.55% of the variance in energy density is explained by the selected predictors. This is a decent explanatory power, but there's room for improvement.

Adjusted R-squared: 0.4405

This metric adjusts for the number of predictors and confirms that the model remains robust with these variables.

F-statistic: 89.49 (p-value < 2.2e-16)

The F-test confirms that the model as a whole is statistically significant (p-value < 0.05), meaning that at least some of the predictors are significantly related to energy density.

Getting the Coefficient Values from the Stepwise Regression

```
# Getting the coefficients from the stepwise model
coef_table <- broom::tidy(stepwise_model)

# Removing the intercept and filter significant terms (p < 0.05)
coef_table_clean <- coef_table %>%
  filter(term != "(Intercept)" & p.value < 0.05) %>%
  arrange(estimate)

# Getting the confidence intervals for the coefficients
conf_intervals <- confint(stepwise_model)[-1, ] # Remove the intercept row

# Ensuring that the number of rows in the confidence intervals matches the number of coefficients
if (nrow(coef_table_clean) != nrow(conf_intervals)) {
  stop("Mismatch in the number of rows between coefficients and confidence intervals.")
}

# Merging the coefficients with their confidence intervals
coef_table_clean$conf.low <- conf_intervals[, 1]
coef_table_clean$conf.high <- conf_intervals[, 2]

# Checking the final table to ensure conf.low and conf.high are added correctly
head(coef_table_clean)
```

```
## # A tibble: 6 x 7
##   term                estimate std.error statistic  p.value conf.low conf.high
##   <chr>                <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
```

```
## 1 household_compositio~ -3.01e-3 1.27e-3 -2.37 1.82e- 2 5.91e-3 1.02e-2
## 2 household_language_2~ -2.84e-3 3.91e-4 -7.26 8.19e-13 -5.50e-3 -5.12e-4
## 3 obesity_percent_of_m~ -1.55e-3 5.20e-4 -2.98 2.93e- 3 2.08e-4 1.70e-3
## 4 income_deprivation_2~ -8.46e-4 2.80e-4 -3.02 2.57e- 3 -3.61e-3 -2.07e-3
## 5 household_income_est~ -3.14e-6 3.72e-7 -8.44 1.24e-16 2.86e-4 7.67e-3
## 6 household_compositio~ 9.56e-4 3.81e-4 2.51 1.23e- 2 -3.87e-6 -2.41e-6
```

Creating the Coefficient Plot For the variables selected by Stepwise regression

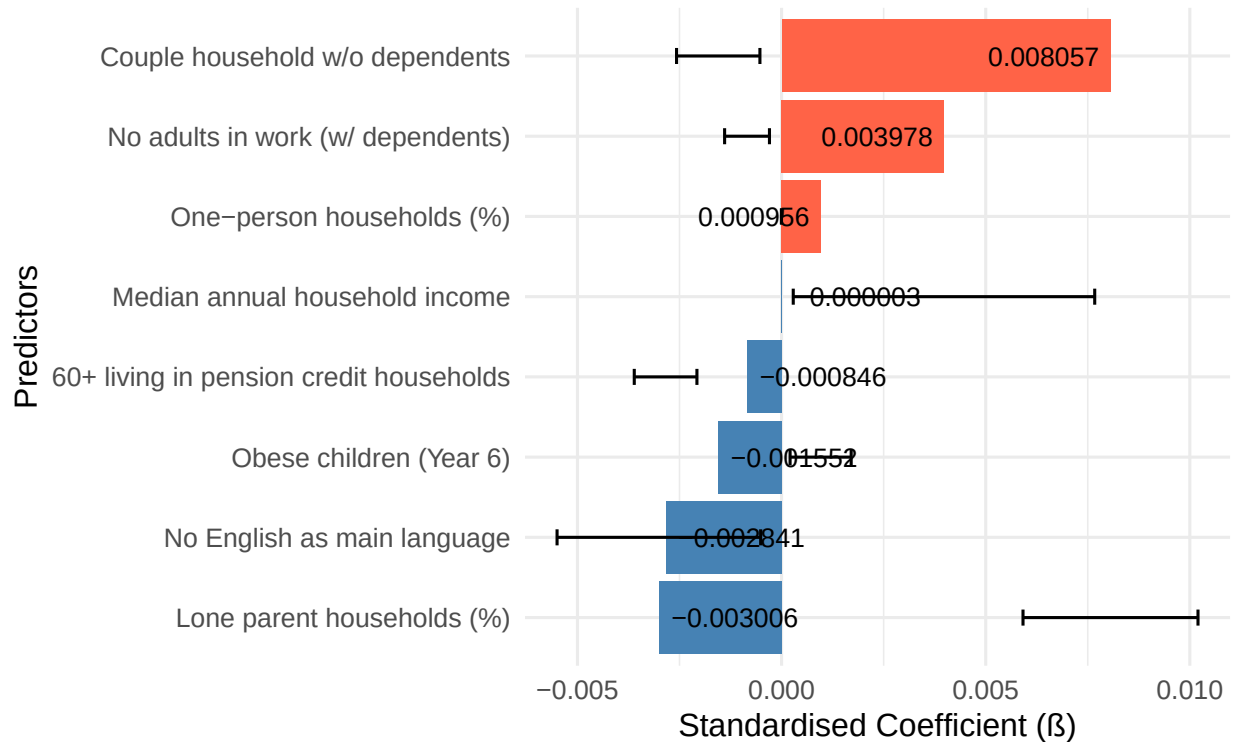
```
# Creating a mapping of long variable names to shorter labels
labels_map <- c(
  "household_composition_2011_percentages_couple_household_without_dependent_children" = "Couple househ",
  "adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependent_ch",
  "household_composition_2011_percentages_one_person_household" = "One-person households (%)",
  "household_income_estimates_2011_12_total_median_annual_household_income_u_fffd" = "Median annual hou",
  "income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households" = "60+",
  "obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12" = "Obese",
  "household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main_la",
  "household_composition_2011_percentages_lone_parent_household" = "Lone parent households (%)"
)

# Applying this mapping to the coefficient data
coef_table_clean$label <- labels_map[coef_table_clean$term]

# Displaying coefficients with more precision (up to 6 decimal places)
coef_table_clean$beta_lbl <- sprintf("%.6f", coef_table_clean$estimate)

# Creating the plot with standard ggplot
ggplot(coef_table_clean, aes(x = reorder(label, estimate), y = estimate,
  ymin = conf.low, ymax = conf.high, fill = estimate > 0)) +
  geom_bar(stat = "identity") +
  geom_errorbar(width = 0.2) +
  # Adding coefficient values beside bars
  geom_text(aes(label = beta_lbl,
    hjust = ifelse(coef_table_clean$estimate > 0, 1.1, -0.1),
    color = "black", size = 3.5) + # Positioning labels to avoid overlap
  coord_flip() +
  labs(title = "Stepwise Regression: Significant Coefficients",
    subtitle = "Standardised Coefficients with 95% Confidence Intervals",
    x = "Predictors", y = "Standardised Coefficient ( )") +
  scale_fill_manual(values = c("steelblue", "tomato")) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none")
```

Stepwise Regression: Significant Coefficients Standardised Coefficients with 95% Confidence Int



```
# Saving the plot to the 'figs' folder
ggsave("figs/stepwise_coefficients_with_values.png", width = 8, height = 6, dpi = 300)
```

Creating the Scatter Plot for the selected variables

```
# Final selected variables after stepwise regression
final_selected_variables <- c(
  "household_composition_2011_percentages_couple_household_without_dependent_children",
  "household_composition_2011_percentages_lone_parent_household",
  "household_composition_2011_percentages_one_person_household",
  "household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main_language",
  "adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependent_children",
  "household_income_estimates_2011_12_total_median_annual_household_income_uk_fffd",
  "income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households",
  "obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12"
)

# Ensuring the 'figs' folder exists
if (!dir.exists("figs")) dir.create("figs", recursive = TRUE)

# Looping through the selected variables and create individual scatter plots
for (var in final_selected_variables) {
  # Create the scatter plot for each variable against energy_density
}
```

```

p <- ggplot(stats_data, aes_string(x = var, y = "energy_density")) +
  geom_point(color = "blue", alpha = 0.6, na.rm = TRUE) + # Blue dots
  geom_smooth(method = "lm", se = TRUE, color = "red") + # Red regression line
  labs(title = paste("Energy Density vs", var),
        x = var, y = "Energy Density (kcal/g)") +
  theme_minimal(base_size = 12)

# Saving the plot in the 'figs' folder with a name corresponding to the variable
ggsave(paste0("figs/scatter_", gsub("[^A-Za-z0-9]", "_", var), ".png"), p, width = 8, height = 6, dpi = 300)

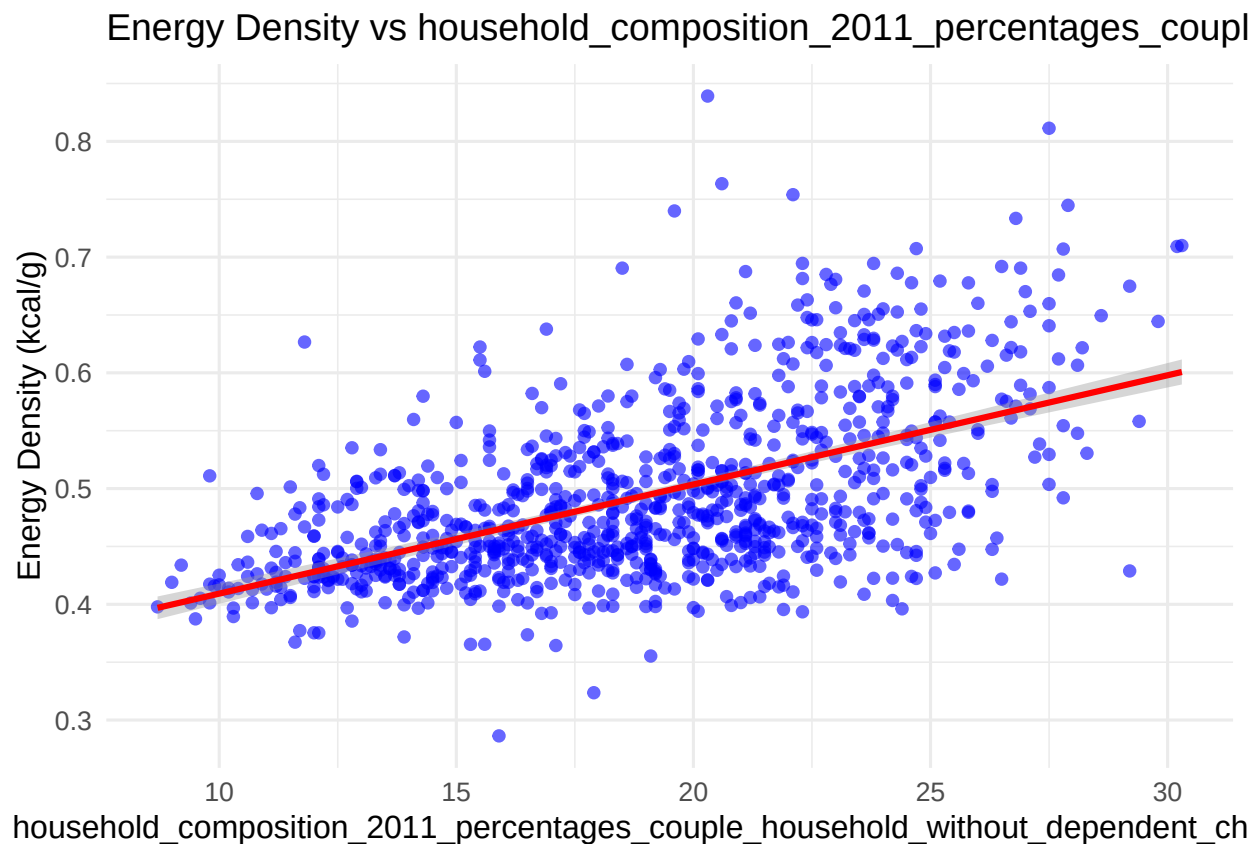
print(p)
}

```

```

## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'

```

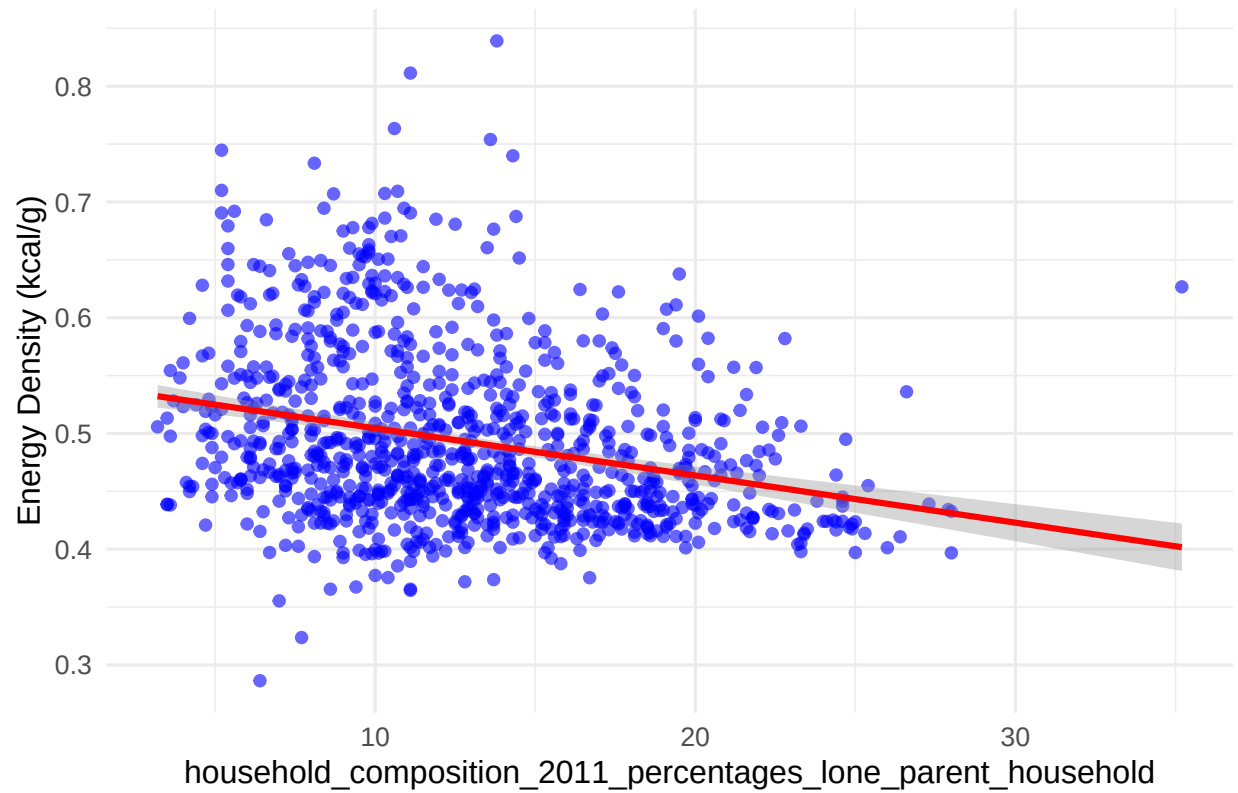


```

## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'

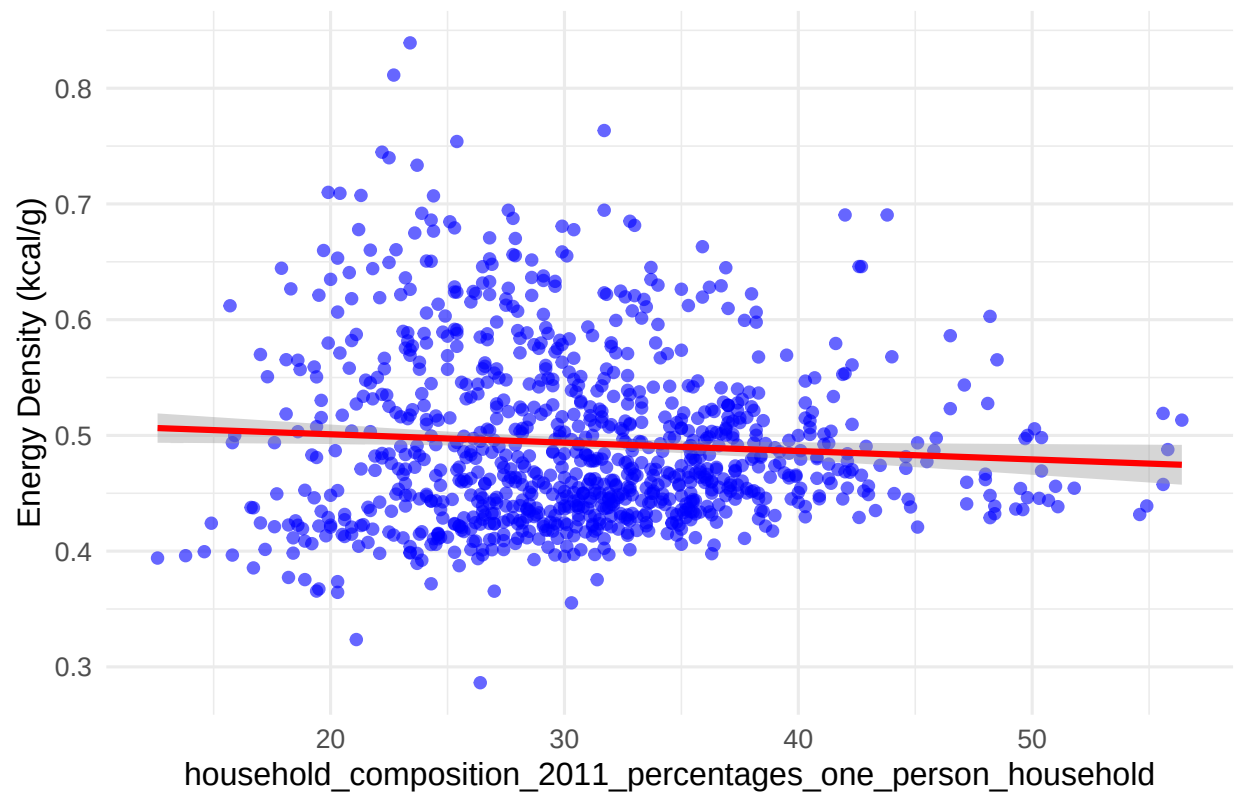
```


Energy Density vs household_composition_2011_percentages_lone_



```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```

Energy Density vs household_composition_2011_percentages_one_p



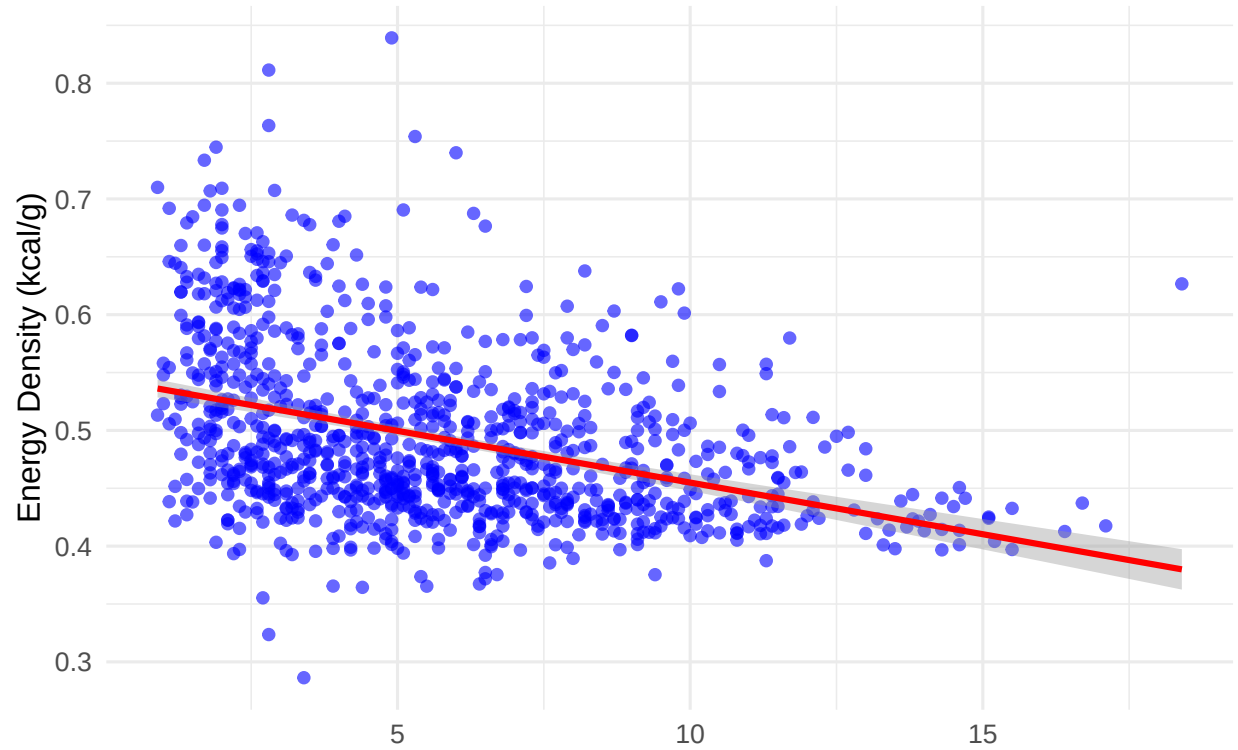
```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```

Energy Density vs household_language_2011_percent_of_household



```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```

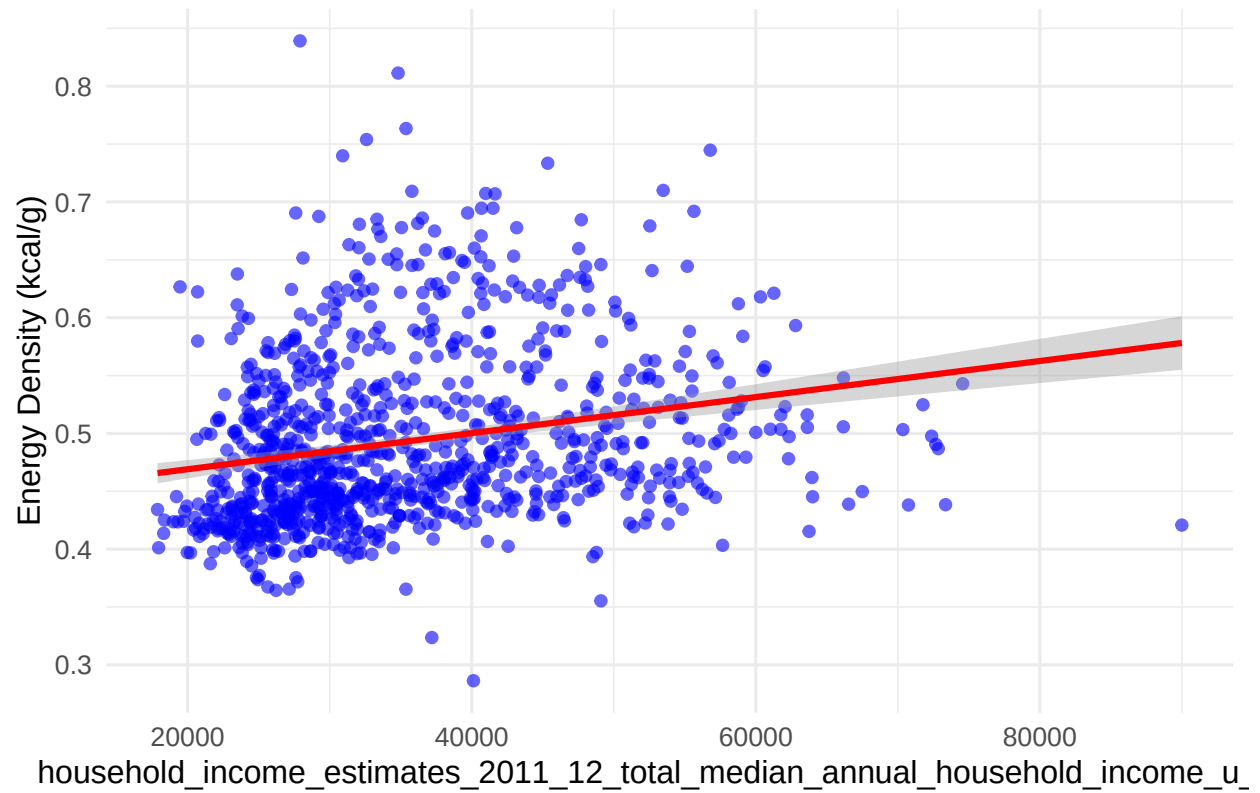
Energy Density vs adults_in_employment_2011_census_percent_of_



ployment_2011_census_percent_of_households_with_no_adults_in_employment_with_

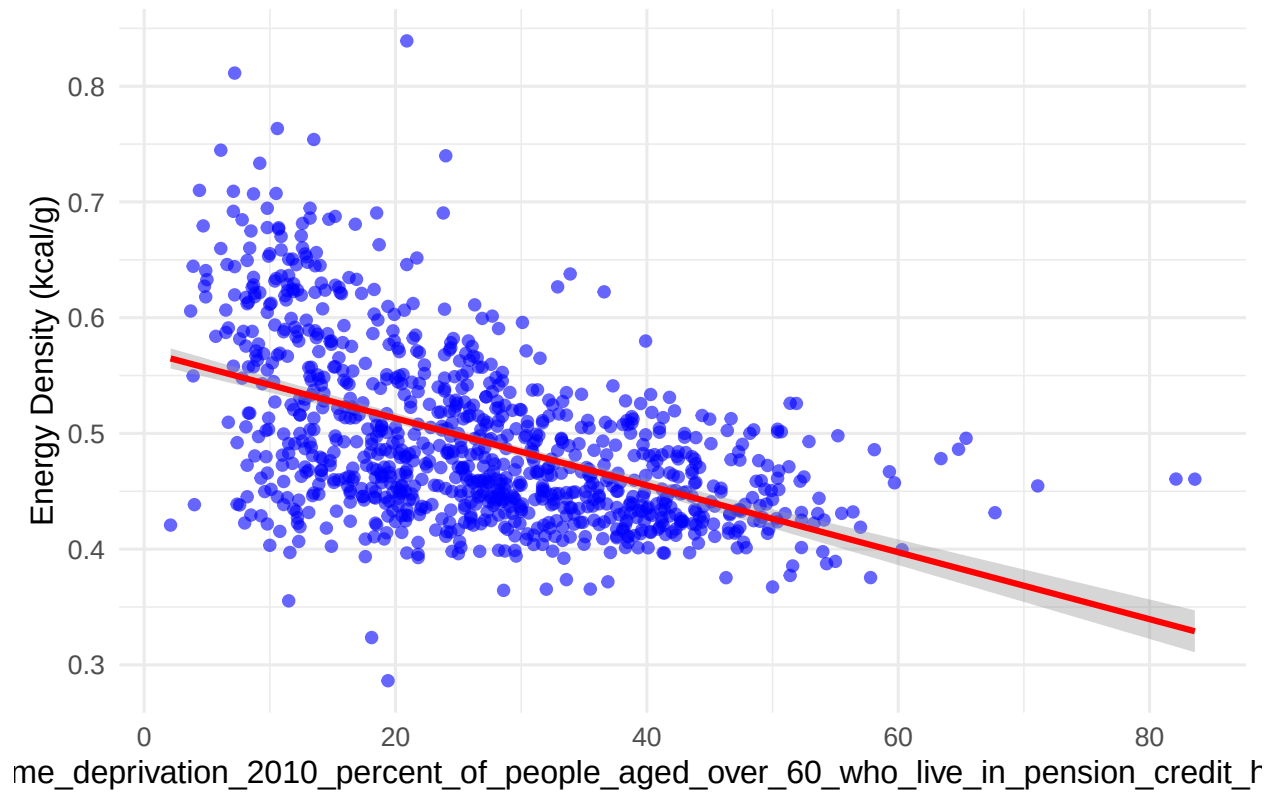
```
## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'
```

Energy Density vs household_income_estimates_2011_12_total_mer



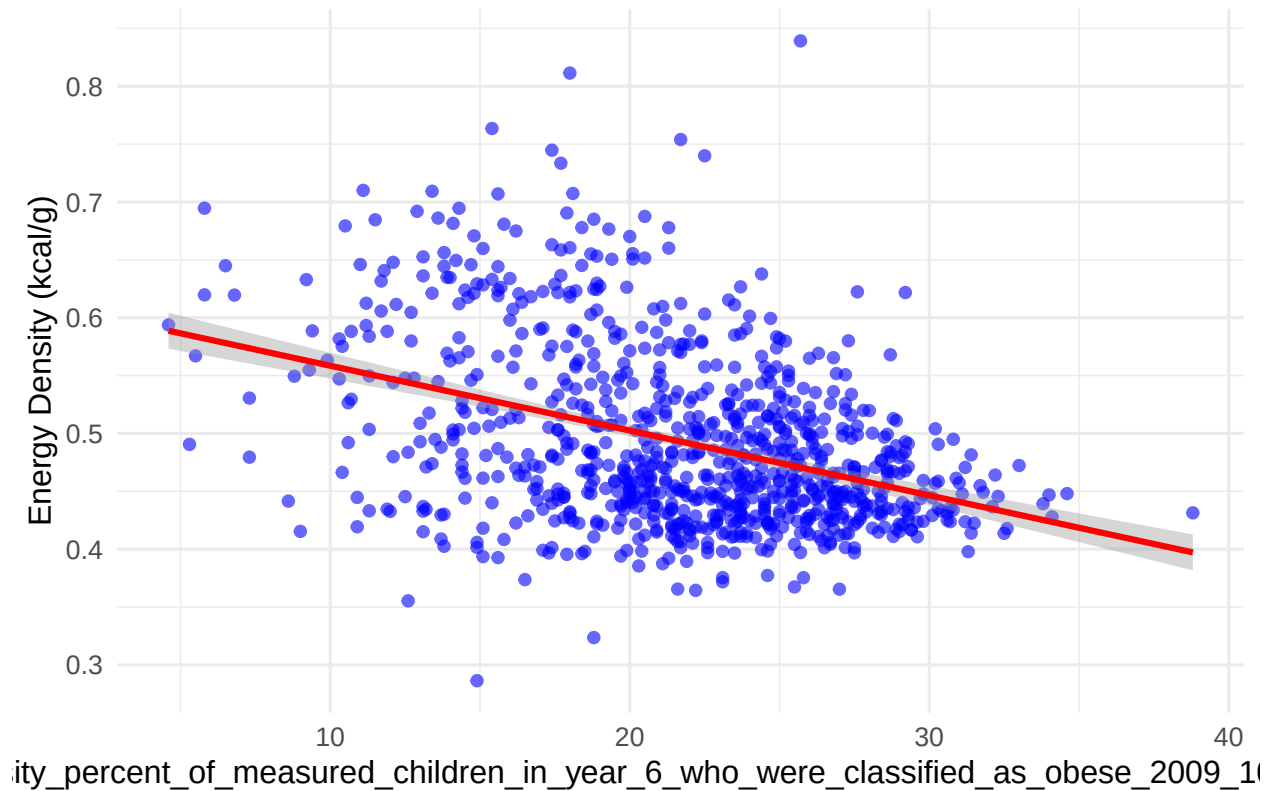
```
## `geom_smooth()` using formula = 'y ~ x'  
## `geom_smooth()` using formula = 'y ~ x'
```

Energy Density vs income_deprivation_2010_percent_of_people_age



```
## `geom_smooth()` using formula = 'y ~ x'
```

Energy Density vs obesity_percent_of_measured_children_in_year_6



Creating Spatial Maps

```
library(tmap)
library(sf)
library(dplyr)

# 1) Final selected vars
final_selected_variables <- c(
  "household_composition_2011_percentages_couple_household_without_dependent_children",
  "household_composition_2011_percentages_lone_parent_household",
  "household_composition_2011_percentages_one_person_household",
  "household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main_language",
  "adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependent_children",
  "household_income_estimates_2011_12_total_median_annual_household_income_upper",
  "income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households",
  "obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12"
)

labels_map <- c(
  "household_composition_2011_percentages_couple_household_without_dependent_children" =
    "Couple hh w/o dependents (%)",
  "household_composition_2011_percentages_lone_parent_household" =
    "Lone-parent hh (%)",
```

```

"household_composition_2011_percentages_one_person_household" =
  "One-person hh (%)",
"household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main_lang" =
  "No English main lang (%)",
"adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependent_children" =
  "No adults in work w/ dependents (%)",
"household_income_estimates_2011_12_total_median_annual_household_income_us_dollars" =
  "Median hh income (£)",
"income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households" =
  "60+ on pension credit (%)",
"obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12" =
  "Year 6 obese (%)"
)

# 2) Ensuring we have an sf and numeric columns (for mapping)
stopifnot(inherits(msoa_all, "sf"))
msoa_all <- msoa_all |>
  mutate(across(all_of(final_selected_variables),
    ~ suppressWarnings(as.numeric(.))))

msoa_all <- msoa_all |>
  mutate(across(all_of(final_selected_variables),
    ~ ifelse(is.na(.), median(., na.rm = TRUE), .)))

# 3) Choosing a vivid palette (exact, valid names) and generate a color vector
if (!requireNamespace("cols4all", quietly = TRUE)) install.packages("cols4all")
library(cols4all)
pal_vals <- c4a("plasma", 5) # <-- change to "viridis", "magma", "carto.tropic", etc. if you prefer

# 4) Map builder (tmap v4.2)
tmap_mode("plot")

```

```
## i tmap modes "plot" - "view"
```

```

if (!dir.exists("figs/maps")) dir.create("figs/maps", recursive = TRUE)

make_map <- function(var) {
  tm_shape(msoa_all) +
    tm_polygons(
      fill = var,
      fill.scale = tm_scale_intervals(
        style = "quantile", # 5 equal-frequency bins
        n = 5,
        values = pal_vals, # pass the actual vector of colors
        value.na = "grey90"
      ),
      fill.legend = tm_legend(
        title = labels_map[[var]],
        frame = TRUE
      )
    ) +
  tm_borders(col = "white", lwd = 0.4) +

```



```

tm_title(paste0(labels_map[[var]], " in London")) +
tm_layout(legend.outside = TRUE, frame = FALSE)
}

# 5) Render + save all maps
for (var in final_selected_variables) {
  if (!var %in% names(msoa_all)) next
  p <- make_map(var)
  print(p)
  tmap_save(
    p,
    filename = paste0("figs/maps/", gsub("[^A-Za-z0-9]+", "_", var), "_map.png"),
    width = 2200, height = 1600, dpi = 220
  )
}

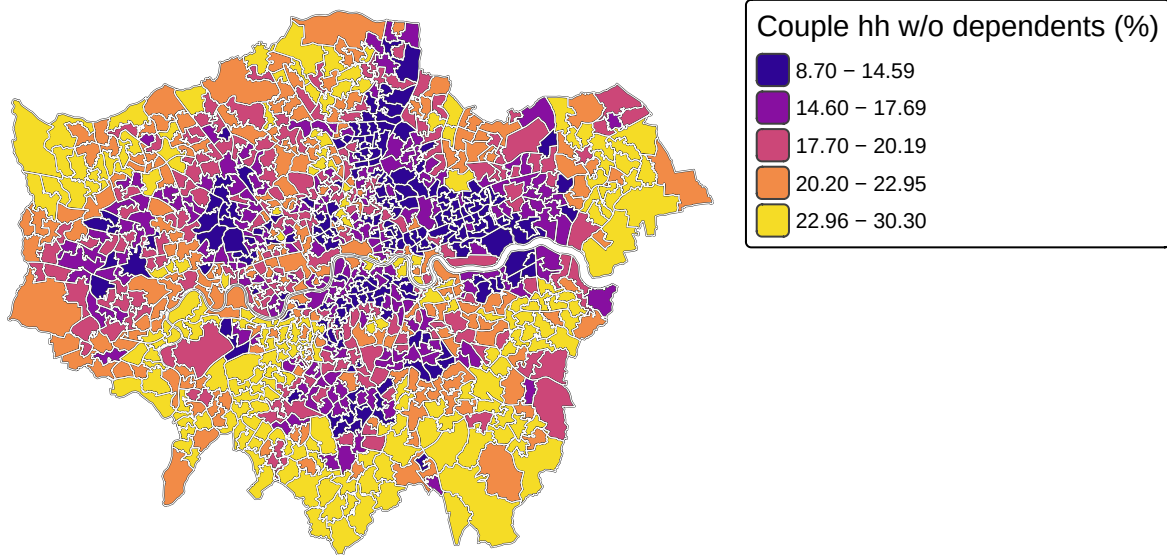
```

```

## Map saved to U:\Research project\figs\maps\household_composition_2011_percentages_couple_household_w
## Resolution: 2200 by 1600 pixels
## Size: 10 by 7.272727 inches (220 dpi)

```

Couple hh w/o dependents (%) in London

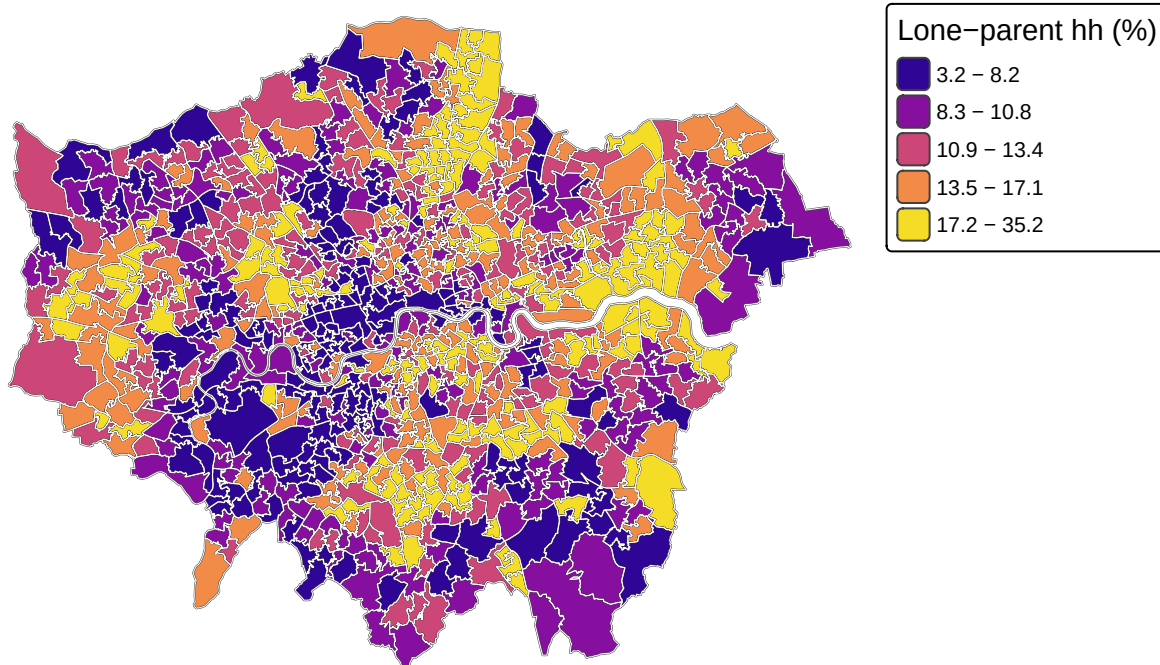


```

## Map saved to U:\Research project\figs\maps\household_composition_2011_percentages_lone_parent_househ
## Resolution: 2200 by 1600 pixels
## Size: 10 by 7.272727 inches (220 dpi)

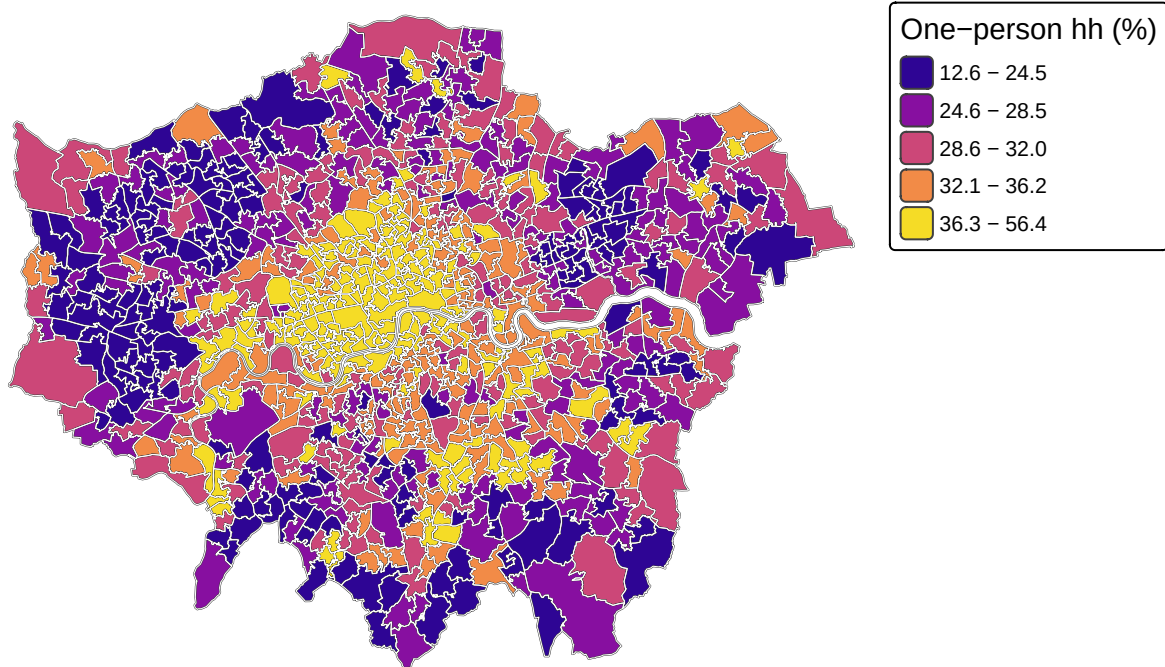
```

Lone-parent hh (%) in London



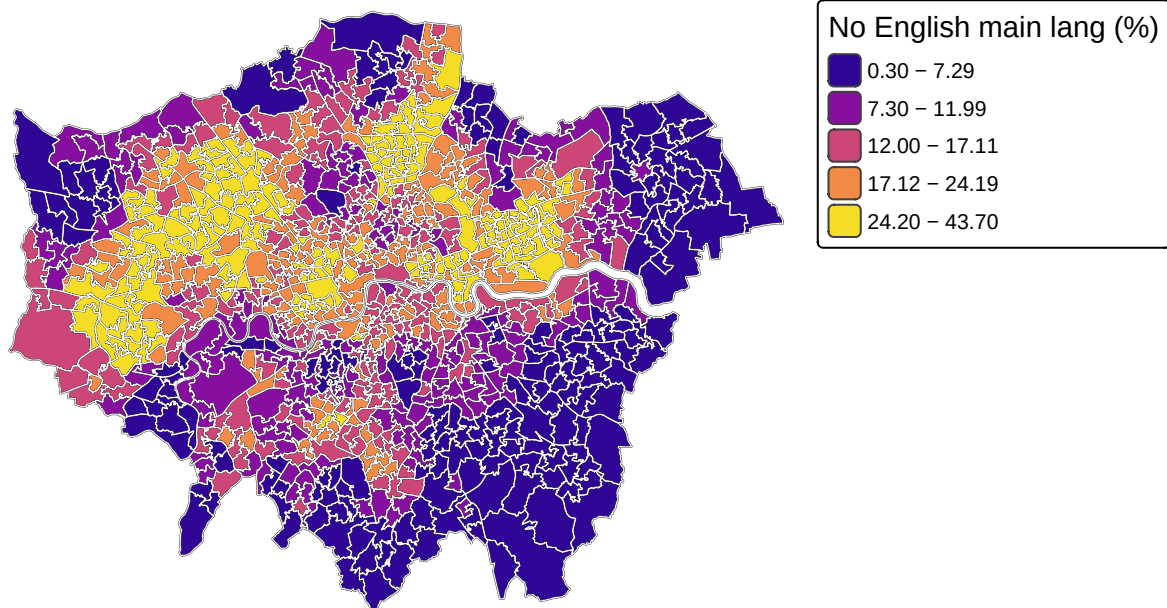
```
## Map saved to U:\Research project\figs\maps\household_composition_2011_percentages_one_person_househo.  
## Resolution: 2200 by 1600 pixels  
## Size: 10 by 7.272727 inches (220 dpi)
```

One-person hh (%) in London



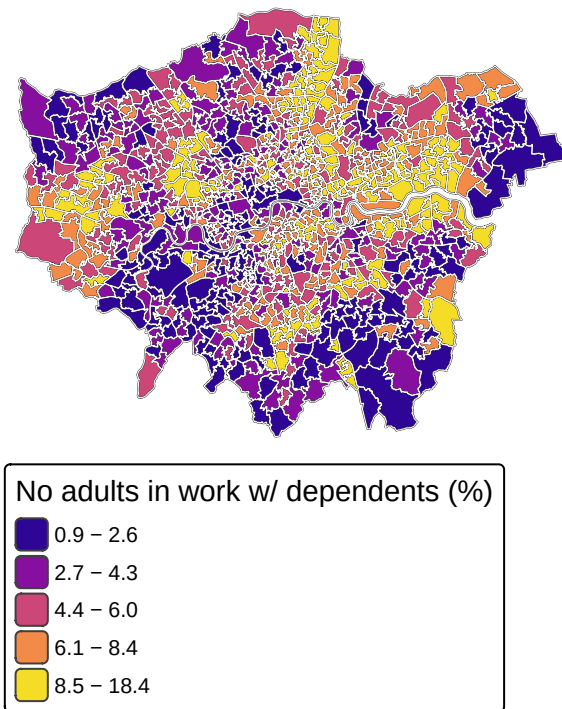
Map saved to U:\Research project\figs\maps\household_language_2011_percent_of_households_where_no_pe
Resolution: 2200 by 1600 pixels
Size: 10 by 7.272727 inches (220 dpi)

No English main lang (%) in London



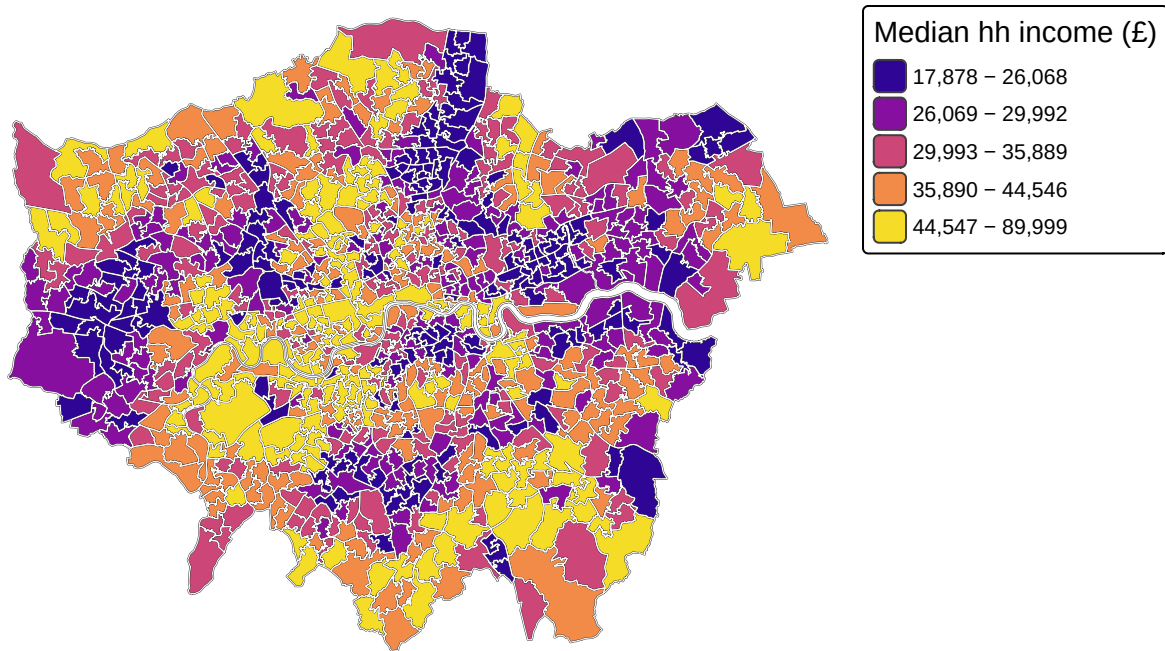
```
## [plot mode] fit legend/component: Some legend items or map components do not
## fit well, and are therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
## Map saved to U:\Research project\figs\maps\adults_in_employment_2011_census_percent_of_households_wi
##
## Resolution: 2200 by 1600 pixels
##
## Size: 10 by 7.272727 inches (220 dpi)
```

No adults in work w/ dependents (%) in London



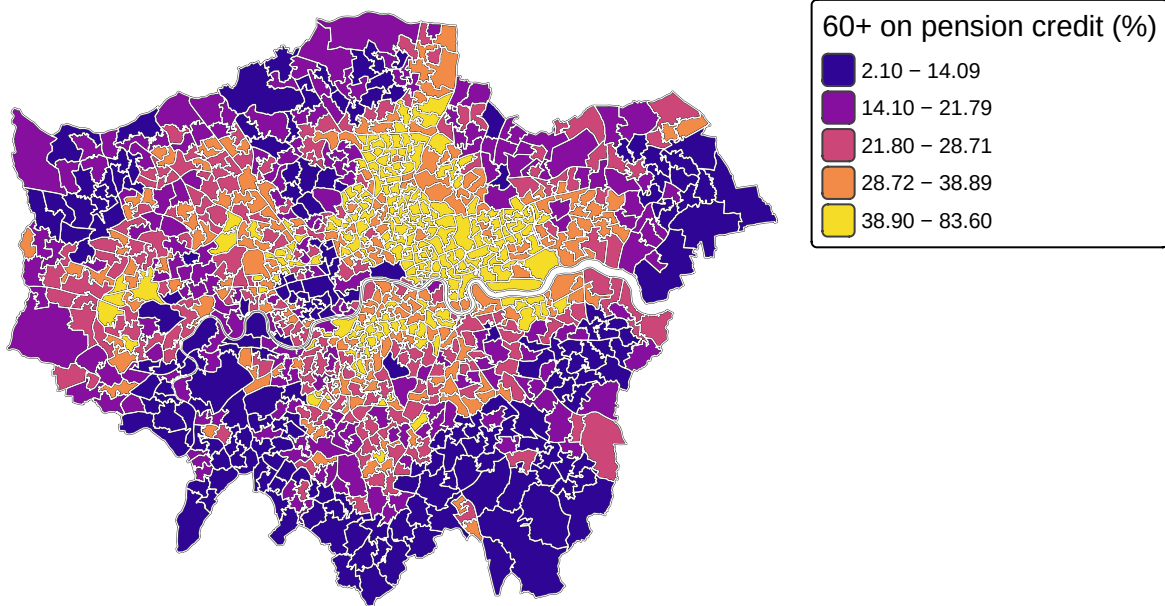
Map saved to U:\Research project\figs\maps\household_income_estimates_2011_12_total_median_annual_ho
Resolution: 2200 by 1600 pixels
Size: 10 by 7.272727 inches (220 dpi)

Median hh income (£) in London



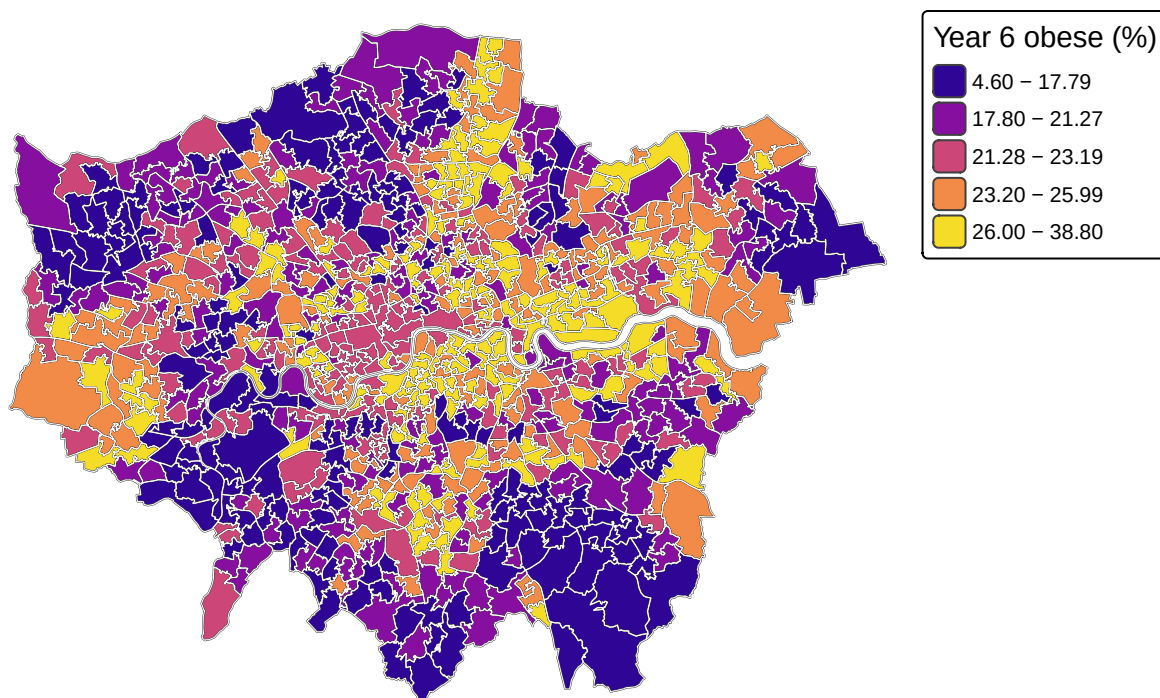
```
## Map saved to U:\Research project\figs\maps\income_deprivation_2010_percent_of_people_aged_over_60_wh  
## Resolution: 2200 by 1600 pixels  
## Size: 10 by 7.272727 inches (220 dpi)
```

60+ on pension credit (%) in London



Map saved to U:\Research project\figs\maps\obesity_percent_of_measured_children_in_year_6_who_were_c
Resolution: 2200 by 1600 pixels
Size: 10 by 7.272727 inches (220 dpi)

Year 6 obese (%) in London



Getting Names of a regions where each varibale is High /Low

```
library(dplyr)
library(readr)
library(sf)

stopifnot(inherits(msoa_all, "sf"))

# Making sure these are numeric for ranking
msoa_all <- msoa_all %>%
  dplyr::mutate(dplyr::across(all_of(final_selected_variables), ~ suppressWarnings(as.numeric(.))))

msoa_name_col <- intersect(names(msoa_all), c("msoa_name", "msoa_name.x", "MSOA11NM", "MSOA21NM"))[1]
lad_name_col <- intersect(names(msoa_all), c("LAD11NM", "LAD21NM", "LAD23NM", "lad_name", "Borough", "LA_NAME"))[1]

if (!dir.exists("tables")) dir.create("tables", recursive = TRUE)

for (var in final_selected_variables) {
  if (!var %in% names(msoa_all)) {
    message("Skipping (missing in data): ", var)
    next
  }
}
```



```

# Building a safe selection vector for reporting columns
cols_keep <- c("msoa_code", msoa_name_col, lad_name_col, var)
cols_keep <- cols_keep[!is.na(cols_keep) & cols_keep %in% names(msoa_all)]

df <- msoa_all %>%
  sf::st_drop_geometry() %>%
  dplyr::select(dplyr::all_of(cols_keep)) %>%
  # Give consistent column names for pretty output
  dplyr::rename(
    msoa      = dplyr::all_of(msoa_name_col),
    borough   = dplyr::all_of(lad_name_col)
  ) %>%
  dplyr::mutate(value = .data[[var]])

# Top 10
top_regions <- df %>%
  dplyr::arrange(dplyr::desc(.data$value)) %>%
  dplyr::slice_head(n = 10)

# Bottom 10
bottom_regions <- df %>%
  dplyr::arrange(.data$value) %>%
  dplyr::slice_head(n = 10)

# Console summary
cat("\n=====\\n")
cat("Variable:", labels_map[[var]] %||% var, "\\n")
cat("Top 10 regions (highest values):\\n"); print(top_regions)
cat("\\nBottom 10 regions (lowest values):\\n"); print(bottom_regions)
cat("=====\\n")

# Save CSVs
safe <- gsub("[^A-Za-z0-9]+", "_", var)
readr::write_csv(top_regions, file.path("tables", paste0("top10_", safe, ".csv")))
readr::write_csv(bottom_regions, file.path("tables", paste0("bottom10_", safe, ".csv")))
}

```

```

##
## =====
## Variable: Couple hh w/o dependents (%)
## Top 10 regions (highest values):
##   msoa_code      msoa   borough
## 1 E02000487 Havering 024 Havering
## 2 E02000165 Bromley 039 Bromley
## 3 E02000861 Sutton 022 Sutton
## 4 E02000478 Havering 015 Havering
## 5 E02000488 Havering 025 Havering
## 6 E02006789 Bromley 042 Bromley
## 7 E02000471 Havering 008 Havering
## 8 E02000699 Merton 011 Merton
## 9 E02000078 Bexley 014 Bexley
## 10 E02000226 Croydon 033 Croydon
##   household_composition_2011_percentages_couple_household_without_dependent_children

```

```

## 1 30.3
## 2 30.2
## 3 29.8
## 4 29.4
## 5 29.2
## 6 29.2
## 7 28.6
## 8 28.3
## 9 28.2
## 10 28.1
## value
## 1 30.3
## 2 30.2
## 3 29.8
## 4 29.4
## 5 29.2
## 6 29.2
## 7 28.6
## 8 28.3
## 9 28.2
## 10 28.1
##
## Bottom 10 regions (lowest values):
## msoa_code msoa borough
## 1 E02000357 Hackney 013 Hackney
## 2 E02000119 Brent 027 Brent
## 3 E02000346 Hackney 002 Hackney
## 4 E02000718 Newham 005 Newham
## 5 E02000732 Newham 019 Newham
## 6 E02000626 Lambeth 009 Lambeth
## 7 E02000117 Brent 025 Brent
## 8 E02000398 Haringey 002 Haringey
## 9 E02000935 Wandsworth 013 Wandsworth
## 10 E02000824 Southwark 018 Southwark
## household_composition_2011_percentages_couple_household_without_dependent_children
## 1 8.7
## 2 9.0
## 3 9.2
## 4 9.4
## 5 9.5
## 6 9.6
## 7 9.8
## 8 9.8
## 9 9.8
## 10 9.9
## value
## 1 8.7
## 2 9.0
## 3 9.2
## 4 9.4
## 5 9.5
## 6 9.6
## 7 9.8
## 8 9.8

```

```

## 9      9.8
## 10     9.9
## =====
##
## =====
## Variable: Lone-parent hh (%)
## Top 10 regions (highest values):
##      msoa_code                msoa                borough
## 1  E02000223                Croydon 030                Croydon
## 2  E02000113                  Brent 021                  Brent
## 3  E02000402                Haringey 006                Haringey
## 4  E02000373 Hammersmith and Fulham 002 Hammersmith and Fulham
## 5  E02000401                Haringey 005                Haringey
## 6  E02006788                Croydon 045                Croydon
## 7  E02000641                Lambeth 024                Lambeth
## 8  E02000398                Haringey 002                Haringey
## 9  E02006783                Lewisham 037                Lewisham
## 10 E02000306                Enfield 030                Enfield
##      household_composition_2011_percentages_lone_parent_household value
## 1                                     35.2  35.2
## 2                                     28.0  28.0
## 3                                     28.0  28.0
## 4                                     27.9  27.9
## 5                                     27.3  27.3
## 6                                     26.6  26.6
## 7                                     26.4  26.4
## 8                                     26.0  26.0
## 9                                     25.4  25.4
## 10                                    25.3  25.3
##
## Bottom 10 regions (lowest values):
##      msoa_code                msoa                borough
## 1  E02000592 Kensington and Chelsea 016 Kensington and Chelsea
## 2  E02000001          City of London 001          City of London
## 3  E02000588 Kensington and Chelsea 012 Kensington and Chelsea
## 4  E02000970          Westminster 011          Westminster
## 5  E02000586 Kensington and Chelsea 010 Kensington and Chelsea
## 6  E02000890          Tower Hamlets 027          Tower Hamlets
## 7  E02000932          Wandsworth 010          Wandsworth
## 8  E02000692          Merton 004          Merton
## 9  E02000931          Wandsworth 009          Wandsworth
## 10 E02000176          Camden 011          Camden
##      household_composition_2011_percentages_lone_parent_household value
## 1                                     3.2  3.2
## 2                                     3.5  3.5
## 3                                     3.5  3.5
## 4                                     3.5  3.5
## 5                                     3.6  3.6
## 6                                     3.6  3.6
## 7                                     3.6  3.6
## 8                                     3.7  3.7
## 9                                     3.9  3.9
## 10                                    4.0  4.0
## =====

```

```

##
## =====
## Variable: One-person hh (%)
## Top 10 regions (highest values):
##      msoa_code          msoa          borough
## 1  E02000001      City of London 001      City of London
## 2  E02000193          Camden 028          Camden
## 3  E02000972      Westminster 013      Westminster
## 4  E02000977      Westminster 018      Westminster
## 5  E02000190          Camden 025          Camden
## 6  E02000590 Kensington and Chelsea 014 Kensington and Chelsea
## 7  E02000589 Kensington and Chelsea 013 Kensington and Chelsea
## 8  E02000588 Kensington and Chelsea 012 Kensington and Chelsea
## 9  E02000582 Kensington and Chelsea 006 Kensington and Chelsea
## 10 E02000974      Westminster 015      Westminster
##      household_composition_2011_percentages_one_person_household value
## 1                                     56.4  56.4
## 2                                     55.8  55.8
## 3                                     55.6  55.6
## 4                                     55.6  55.6
## 5                                     54.9  54.9
## 6                                     54.6  54.6
## 7                                     51.8  51.8
## 8                                     51.1  51.1
## 9                                     51.0  51.0
## 10                                    50.7  50.7
##
## Bottom 10 regions (lowest values):
##      msoa_code          msoa          borough
## 1  E02000442      Harrow 010      Harrow
## 2  E02006882      Harrow 033      Harrow
## 3  E02000723      Newham 010      Newham
## 4  E02000440      Harrow 008      Harrow
## 5  E02000231      Croydon 038      Croydon
## 6  E02000254      Ealing 017      Ealing
## 7  E02000728      Newham 015      Newham
## 8  E02000274      Ealing 037      Ealing
## 9  E02000773 Redbridge 023 Redbridge
## 10 E02000724      Newham 011      Newham
##      household_composition_2011_percentages_one_person_household value
## 1                                     12.6  12.6
## 2                                     13.8  13.8
## 3                                     14.6  14.6
## 4                                     14.9  14.9
## 5                                     15.7  15.7
## 6                                     15.8  15.8
## 7                                     15.8  15.8
## 8                                     15.9  15.9
## 9                                     16.6  16.6
## 10                                    16.7  16.7
## =====
##
## =====
## Variable: No English main lang (%)

```

Top 10 regions (highest values):

##	msoa_code	msoa	borough
## 1	E02000274	Ealing 037	Ealing
## 2	E02000263	Ealing 026	Ealing
## 3	E02000728	Newham 015	Newham
## 4	E02000121	Brent 029	Brent
## 5	E02000727	Newham 014	Newham
## 6	E02000731	Newham 018	Newham
## 7	E02000723	Newham 010	Newham
## 8	E02000974	Westminster 015	Westminster
## 9	E02000721	Newham 008	Newham
## 10	E02000260	Ealing 023	Ealing

household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main.

1
2
3
4
5
6
7
8
9
10

value

1 43.7
2 43.4
3 42.3
4 40.4
5 40.0
6 39.1
7 38.5
8 37.9
9 37.5
10 37.3

##

Bottom 10 regions (lowest values):

##	msoa_code	msoa	borough
## 1	E02006789	Bromley 042	Bromley
## 2	E02000487	Havering 024	Havering
## 3	E02000165	Bromley 039	Bromley
## 4	E02000160	Bromley 034	Bromley
## 5	E02000088	Bexley 024	Bexley
## 6	E02000153	Bromley 027	Bromley
## 7	E02000163	Bromley 037	Bromley
## 8	E02000236	Croydon 043	Croydon
## 9	E02000471	Havering 008	Havering
## 10	E02000481	Havering 018	Havering

household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main.

1
2
3
4
5
6

```

## 7
## 8
## 9
## 10
##      value
## 1      0.3
## 2      0.5
## 3      0.9
## 4      1.1
## 5      1.2
## 6      1.2
## 7      1.3
## 8      1.3
## 9      1.4
## 10     1.4
## =====
##
## =====
## Variable: No adults in work w/ dependents (%)
## Top 10 regions (highest values):
##      msoa_code          msoa          borough
## 1 E02000223          Croydon 030          Croydon
## 2 E02000022 Barking and Dagenham 021 Barking and Dagenham
## 3 E02000282          Enfield 006          Enfield
## 4 E02000783          Redbridge 033          Redbridge
## 5 E02000290          Enfield 014          Enfield
## 6 E02000402          Haringey 006          Haringey
## 7 E02000023 Barking and Dagenham 022 Barking and Dagenham
## 8 E02006793          Enfield 037          Enfield
## 9 E02006794          Haringey 037          Haringey
## 10 E02000278          Enfield 002          Enfield
##      adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependen
## 1
## 2
## 3
## 4
## 5
## 6
## 7
## 8
## 9
## 10
##      value
## 1     18.4
## 2     17.1
## 3     16.7
## 4     16.4
## 5     15.5
## 6     15.5
## 7     15.2
## 8     15.1
## 9     15.1
## 10    14.7
##

```

```

## Bottom 10 regions (lowest values):
##      msoa_code                msoa                borough
## 1  E02000001      City of London 001      City of London
## 2  E02000487      Havering 024      Havering
## 3  E02000176      Camden 011      Camden
## 4  E02000478      Havering 015      Havering
## 5  E02000931      Wandsworth 009      Wandsworth
## 6  E02000138      Bromley 012      Bromley
## 7  E02000588 Kensington and Chelsea 012 Kensington and Chelsea
## 8  E02000592 Kensington and Chelsea 016 Kensington and Chelsea
## 9  E02000932      Wandsworth 010      Wandsworth
## 10 E02006836      Sutton 025      Sutton
##      adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependen
## 1
## 2
## 3
## 4
## 5
## 6
## 7
## 8
## 9
## 10
##      value
## 1      0.9
## 2      0.9
## 3      1.0
## 4      1.0
## 5      1.0
## 6      1.1
## 7      1.1
## 8      1.1
## 9      1.1
## 10     1.1
## =====
##
## =====
## Variable: Median hh income (£)
## Top 10 regions (highest values):
##      msoa_code                msoa                borough
## 1  E02000978      Westminster 019      Westminster
## 2  E02000690      Merton 002      Merton
## 3  E02000588 Kensington and Chelsea 012 Kensington and Chelsea
## 4  E02000169      Camden 004      Camden
## 5  E02000939      Wandsworth 017      Wandsworth
## 6  E02000890      Tower Hamlets 027      Tower Hamlets
## 7  E02000167      Camden 002      Camden
## 8  E02000586 Kensington and Chelsea 010 Kensington and Chelsea
## 9  E02000837      Southwark 031      Southwark
## 10 E02000584 Kensington and Chelsea 008 Kensington and Chelsea
##      household_income_estimates_2011_12_total_median_annual_household_income_u_fffd
## 1
## 2
## 3

```

```

## 4 72851
## 5 72680
## 6 72382
## 7 71771
## 8 70752
## 9 70356
## 10 67500
## value
## 1 89999
## 2 74567
## 3 73363
## 4 72851
## 5 72680
## 6 72382
## 7 71771
## 8 70752
## 9 70356
## 10 67500
##
## Bottom 10 regions (lowest values):
## msoa_code msoa borough
## 1 E02000373 Hammersmith and Fulham 002 Hammersmith and Fulham
## 2 E02000398 Haringey 002 Haringey
## 3 E02000306 Enfield 030 Enfield
## 4 E02006794 Haringey 037 Haringey
## 5 E02000408 Haringey 012 Haringey
## 6 E02000577 Kensington and Chelsea 001 Kensington and Chelsea
## 7 E02000049 Barnet 026 Barnet
## 8 E02000223 Croydon 030 Croydon
## 9 E02006793 Enfield 037 Enfield
## 10 E02000402 Haringey 006 Haringey
## household_income_estimates_2011_12_total_median_annual_household_income_u_fffd
## 1 17878
## 2 17955
## 3 18308
## 4 18362
## 5 18993
## 6 19212
## 7 19305
## 8 19464
## 9 19714
## 10 19793
## value
## 1 17878
## 2 17955
## 3 18308
## 4 18362
## 5 18993
## 6 19212
## 7 19305
## 8 19464
## 9 19714
## 10 19793
## =====

```



```

##
## =====
## Variable: 60+ on pension credit (%)
## Top 10 regions (highest values):
##      msoa_code          msoa          borough
## 1  E02000884 Tower Hamlets 021 Tower Hamlets
## 2  E02000878 Tower Hamlets 015 Tower Hamlets
## 3  E02000876 Tower Hamlets 013 Tower Hamlets
## 4  E02000874 Tower Hamlets 011 Tower Hamlets
## 5  E02000877 Tower Hamlets 014 Tower Hamlets
## 6  E02000885 Tower Hamlets 022 Tower Hamlets
## 7  E02000882 Tower Hamlets 019 Tower Hamlets
## 8  E02000723          Newham 010          Newham
## 9  E02000872 Tower Hamlets 009 Tower Hamlets
## 10 E02000880 Tower Hamlets 017 Tower Hamlets
##      income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households
## 1  83.6
## 2  82.1
## 3  71.1
## 4  67.7
## 5  65.4
## 6  64.8
## 7  63.4
## 8  60.3
## 9  59.7
## 10 59.3
##      value
## 1  83.6
## 2  82.1
## 3  71.1
## 4  67.7
## 5  65.4
## 6  64.8
## 7  63.4
## 8  60.3
## 9  59.7
## 10 59.3
##
## Bottom 10 regions (lowest values):
##      msoa_code          msoa          borough
## 1  E02000978          Westminster 019 Westminster
## 2  E02000159          Bromley 033 Bromley
## 3  E02000147          Bromley 021 Bromley
## 4  E02000861          Sutton 022 Sutton
## 5  E02000588 Kensington and Chelsea 012 Kensington and Chelsea
## 6  E02000487          Havering 024 Havering
## 7  E02000482          Havering 019 Havering
## 8  E02000087          Bexley 023 Bexley
## 9  E02000133          Bromley 007 Bromley
## 10 E02000151          Bromley 025 Bromley
##      income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households
## 1  2.1
## 2  3.7
## 3  3.9

```

```

## 4 3.9
## 5 4.0
## 6 4.4
## 7 4.7
## 8 4.8
## 9 4.9
## 10 4.9
## value
## 1 2.1
## 2 3.7
## 3 3.9
## 4 3.9
## 5 4.0
## 6 4.4
## 7 4.7
## 8 4.8
## 9 4.9
## 10 4.9
## =====
##
## =====
## Variable: Year 6 obese (%)
## Top 10 regions (highest values):
## msoa_code msoa borough
## 1 E02000819 Southwark 013 Southwark
## 2 E02000810 Southwark 004 Southwark
## 3 E02000361 Hackney 017 Hackney
## 4 E02000815 Southwark 009 Southwark
## 5 E02000828 Southwark 022 Southwark
## 6 E02000406 Haringey 010 Haringey
## 7 E02000407 Haringey 011 Haringey
## 8 E02000306 Enfield 030 Enfield
## 9 E02000814 Southwark 008 Southwark
## 10 E02000686 Lewisham 034 Lewisham
## obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12
## 1 38.8
## 2 34.6
## 3 34.1
## 4 34.0
## 5 33.8
## 6 33.0
## 7 32.6
## 8 32.5
## 9 32.3
## 10 32.2
## value
## 1 38.8
## 2 34.6
## 3 34.1
## 4 34.0
## 5 33.8
## 6 33.0
## 7 32.6
## 8 32.5

```

```

## 9    32.3
## 10   32.2
##
## Bottom 10 regions (lowest values):
##      msoa_code                msoa                borough
## 1  E02000799 Richmond upon Thames 016 Richmond upon Thames
## 2  E02000939 Wandsworth 017 Wandsworth
## 3  E02000785 Richmond upon Thames 002 Richmond upon Thames
## 4  E02000801 Richmond upon Thames 018 Richmond upon Thames
## 5  E02000802 Richmond upon Thames 019 Richmond upon Thames
## 6  E02000805 Richmond upon Thames 022 Richmond upon Thames
## 7  E02000804 Richmond upon Thames 021 Richmond upon Thames
## 8  E02000429 Haringey 033 Haringey
## 9  E02000699 Merton 011 Merton
## 10 E02000410 Haringey 014 Haringey
##      obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12
## 1  4.6
## 2  5.3
## 3  5.5
## 4  5.8
## 5  5.8
## 6  6.5
## 7  6.8
## 8  7.3
## 9  7.3
## 10 8.6
##      value
## 1    4.6
## 2    5.3
## 3    5.5
## 4    5.8
## 5    5.8
## 6    6.5
## 7    6.8
## 8    7.3
## 9    7.3
## 10   8.6
## =====

```

Creating side-by-side maps.

- This script creates two types of maps for each selected variable in the dataset a full map showing the value of the variable across all London MSOAs, and a high only map that highlights only the top 10 percent of areas while keeping the rest of London in light grey.
- To do this, the code first converts all selected variables to numeric, defines a consistent colour palette and breakpoints, and calculates the 90th-percentile threshold for what counts as high. It then generates each full map using quantile-based colour breaks and an identical colour scale.
- For the high-only version, the script keeps the original values only for MSOAs above the threshold and blanks out the rest so only the highest areas are coloured. Finally, for every variable, the code arranges the full map and the high-only map side-by-side, displays them, and saves each combined figure as a PNG file in the figs/maps folder.

```

library(tmap)
library(sf)
library(dplyr)
library(cols4all)

final_selected_variables <- c(
  "household_composition_2011_percentages_couple_household_without_dependent_children",
  "household_composition_2011_percentages_lone_parent_household",
  "household_composition_2011_percentages_one_person_household",
  "household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main_language",
  "adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependent_children",
  "household_income_estimates_2011_12_total_median_annual_household_income_u_fffd",
  "income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households",
  "obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12"
)

labels_map <- c(
  "household_composition_2011_percentages_couple_household_without_dependent_children" = "Couple hh w/o dependent children (%)",
  "household_composition_2011_percentages_lone_parent_household" = "Lone-parent hh (%)",
  "household_composition_2011_percentages_one_person_household" = "One-person hh (%)",
  "household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main_language" = "English as a main language (%)",
  "adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependent_children" = "Adults in employment (%)",
  "household_income_estimates_2011_12_total_median_annual_household_income_u_fffd" = "Median hh income",
  "income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households" = "60+ in pension credit (%)",
  "obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12" = "Year 6 obesity"
)

# making sure numeric
stopifnot(inherits(msoa_all, "sf"))
msoa_all <- msoa_all %>%
  mutate(across(all_of(final_selected_variables), ~ suppressWarnings(as.numeric(.))))

# same palette as main maps
pal_vals <- c4a("plasma", 5)

# top 10% threshold
HIGH_QUANTILE <- 0.90

tmap_mode("plot")

## i tmap modes "plot" - "view"

if (!dir.exists("figs/maps")) dir.create("figs/maps", TRUE)

make_full_map <- function(var) {
  # breaks based on FULL data range
  brks <- quantile(msoa_all[[var]], probs = seq(0, 1, length.out = 6), na.rm = TRUE)

  tm_shape(msoa_all) +
    tm_polygons(
      fill = var,

```

```

    fill.scale = tm_scale_intervals(
      style = "fixed",      # we now fix breaks explicitly
      breaks = brks,
      values = pal_vals,
      value.na = "grey90"
    ),
    fill.legend = tm_legend(
      title = labels_map[[var]],
      frame = TRUE
    )
  ) +
  tm_borders(col = "white", lwd = 0.4) +
  tm_title(paste0(labels_map[[var]], " in London")) +
  tm_layout(legend.outside = TRUE, frame = FALSE)
}

# -----
# 2) High-only map using SAME scale
# -----
make_high_only_map <- function(var) {
  stopifnot(var %in% names(msoa_all))

  # same global breaks as full map
  brks <- quantile(msoa_all[[var]], probs = seq(0, 1, length.out = 6), na.rm = TRUE)

  # threshold for "high" (top 10%)
  thr <- quantile(msoa_all[[var]], probs = HIGH_QUANTILE, na.rm = TRUE)

  high_col <- paste0(var, "__high_only")

  # keep original values for high areas, NA elsewhere
  msoa_plot <- msoa_all %>%
    mutate(!high_col := ifelse(!is.na(.data[[var]]) & .data[[var]] >= thr,
                               .data[[var]], NA_real_))

  tm_shape(msoa_all) +
    # base layer: all MSOAs in light grey
    tm_polygons(col = "grey90", lwd = 0.2) +
    # overlay: only high areas coloured, with SAME breaks + colours
    tm_shape(msoa_plot) +
    tm_polygons(
      fill = high_col,
      fill.scale = tm_scale_intervals(
        style = "fixed",
        breaks = brks,
        values = pal_vals,
        value.na = NA      # non-high = transparent
      ),
      fill.legend = tm_legend(
        title = paste0(labels_map[[var]], " ( ", round(thr, 2), ")"),
        frame = TRUE
      ),
    ),
    col = NA

```

```

) +
tm_borders(col = "white", lwd = 0.3) +
tm_title(paste0("High areas: ", labels_map[[var]],
               " (Top ", round((1 - HIGH_QUANTILE) * 100), "%)")) +
tm_layout(legend.outside = TRUE, frame = FALSE)
}

# -----
# 3) Side-by-side maps + save
# -----
for (var in final_selected_variables) {
  full_map <- make_full_map(var)
  high_map <- make_high_only_map(var)

  # view in R
  print(tmap_arrange(full_map, high_map, ncol = 2))

  # save combined figure
  safe <- gsub("[^A-Za-z0-9]+", "_", var)
  tmap_save(
    tmap_arrange(full_map, high_map, ncol = 2),
    filename = paste0("figs/maps/combined_", safe, ".png"),
    width = 3000, height = 1600, dpi = 220
  )
}

```

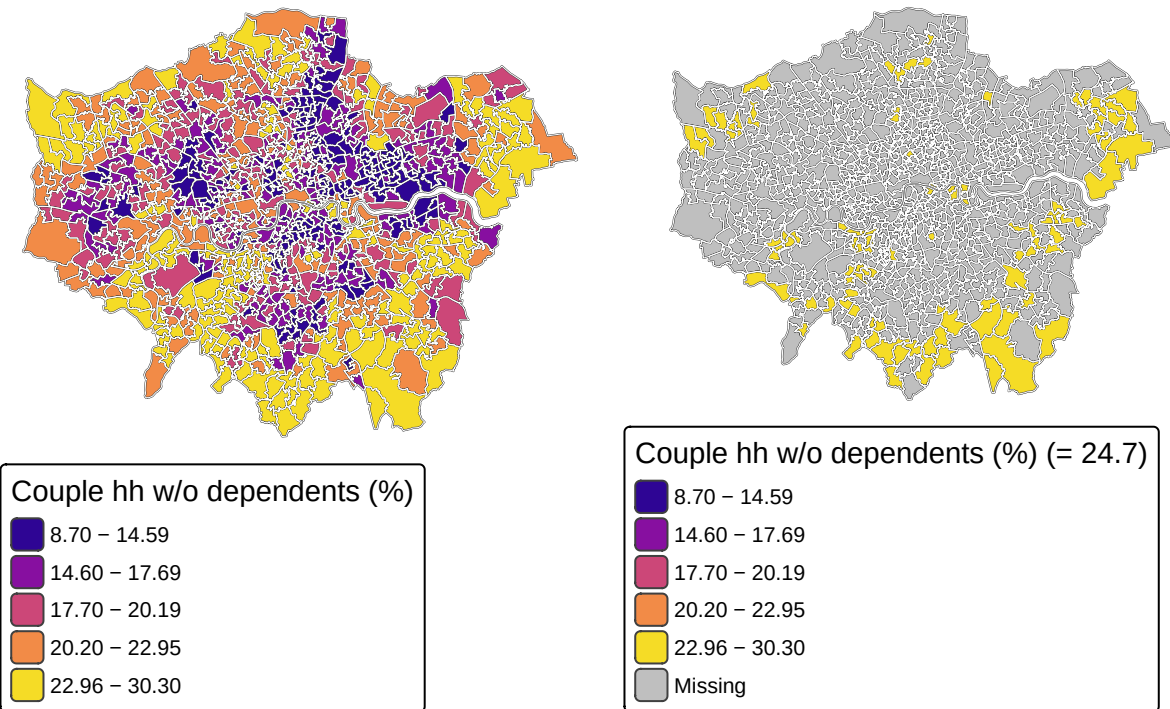
```

## [plot mode] fit legend/component: Some legend items or map components do not
## fit well, and are therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
## [plot mode] fit legend/component: Some legend items or map components do not
## fit well, and are therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
## Map saved to U:\Research project\figs\maps\combined_household_composition_2011_percentages_couple_ho
##
## Resolution: 3000 by 1600 pixels
##
## Size: 13.63636 by 7.272727 inches (220 dpi)

```

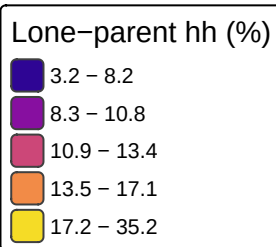
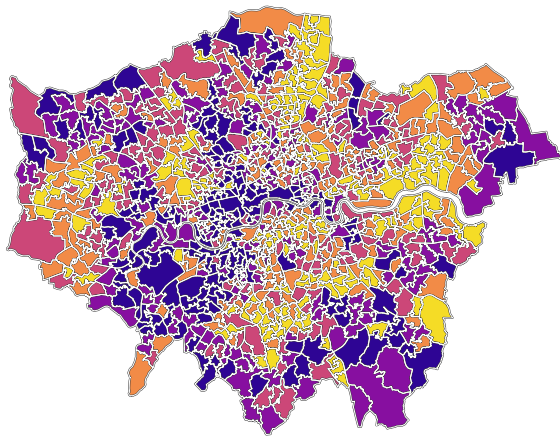
Couple hh w/o dependents (%) in London

High areas: Couple hh w/o dependents (%) (Top 10%)

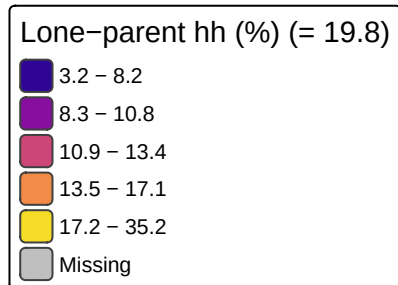
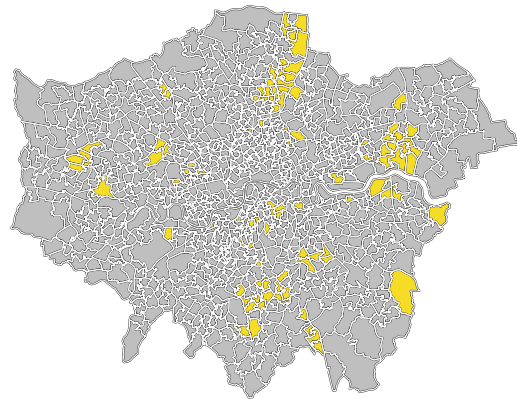


```
## [plot mode] fit legend/component: Some legend items or map components do not
## fit well, and are therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
## Map saved to U:\Research project\figs\maps\combined_household_composition_2011_percentages_lone_pare
##
## Resolution: 3000 by 1600 pixels
##
## Size: 13.63636 by 7.272727 inches (220 dpi)
```

Lone-parent hh (%) in London



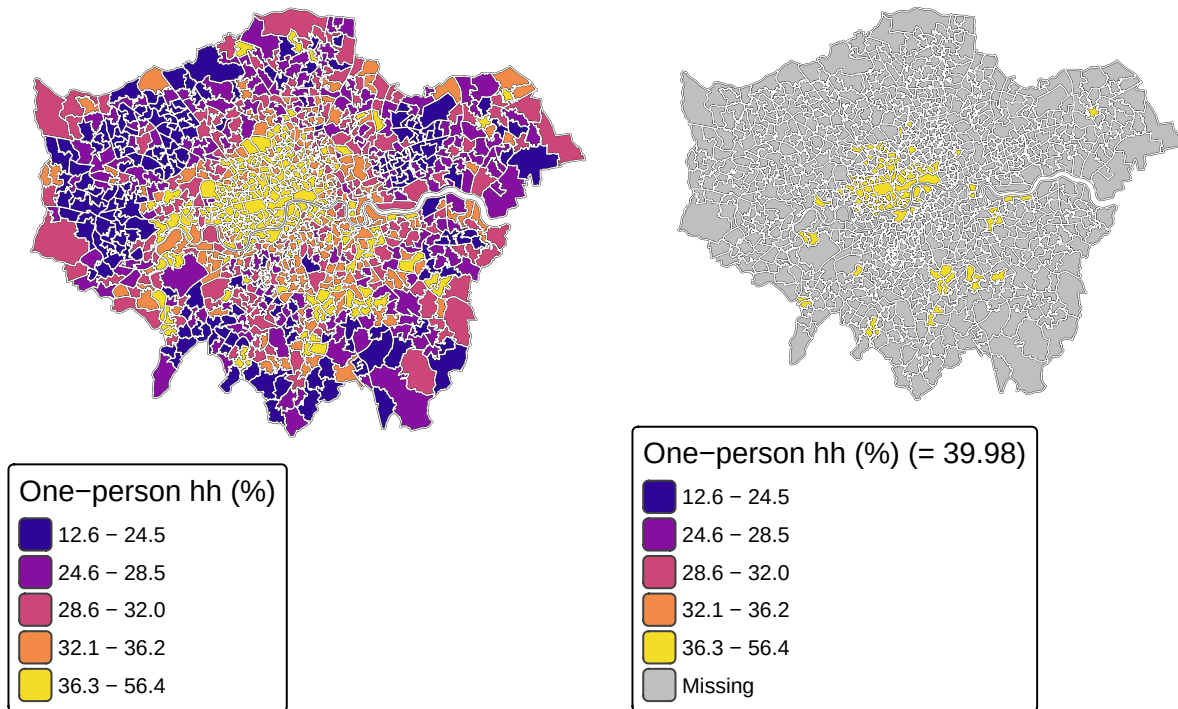
High areas: Lone-parent hh (%) (Top 10%)



```
## [plot mode] fit legend/component: Some legend items or map components do not
## fit well, and are therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
## Map saved to U:\Research project\figs\maps\combined_household_composition_2011_percentages_one_person
##
## Resolution: 3000 by 1600 pixels
##
## Size: 13.63636 by 7.272727 inches (220 dpi)
```


One-person hh (%) in London

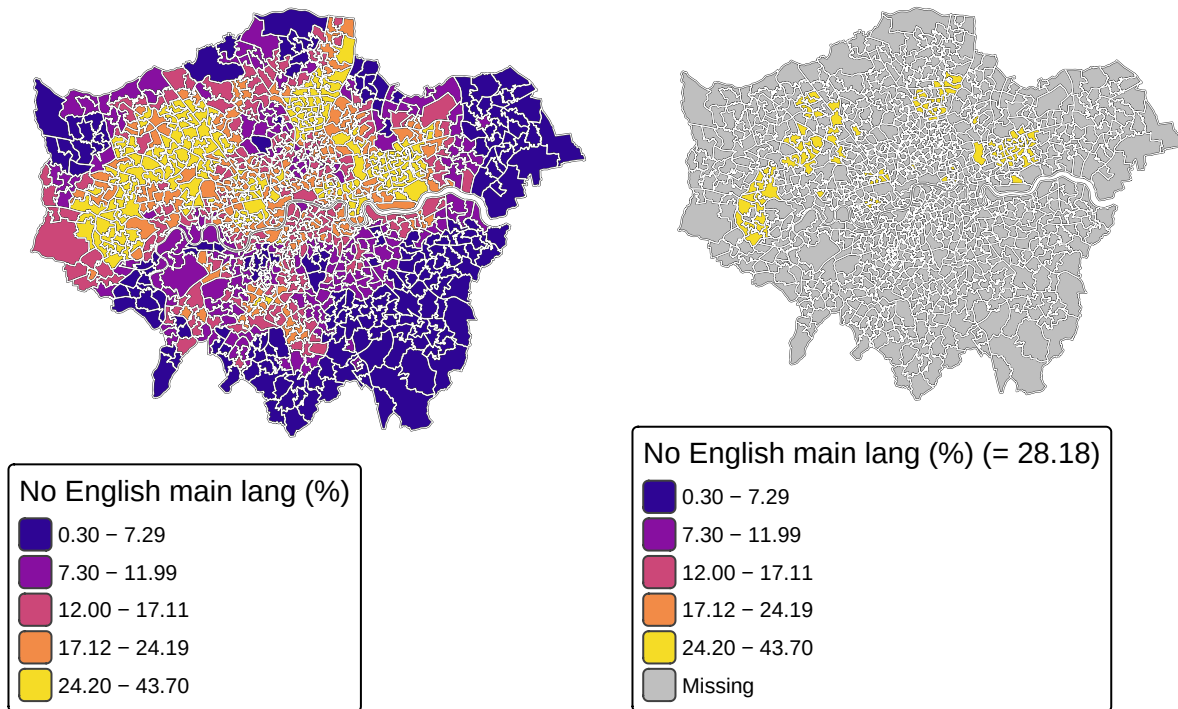
High areas: One-person hh (%) (Top 10%)



```
## [plot mode] fit legend/component: Some legend items or map components do not
## fit well, and are therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
## [plot mode] fit legend/component: Some legend items or map components do not
## fit well, and are therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
## Map saved to U:\Research project\figs\maps\combined_household_language_2011_percent_of_households_who
##
## Resolution: 3000 by 1600 pixels
##
## Size: 13.63636 by 7.272727 inches (220 dpi)
```

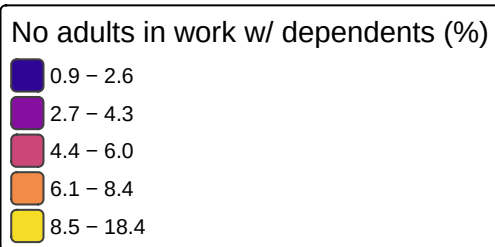
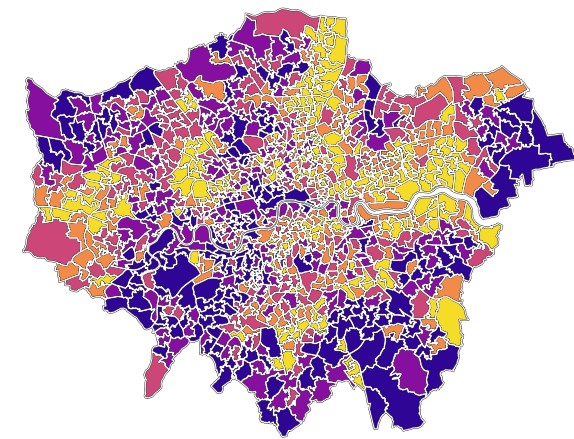
No English main lang (%) in London

High areas: No English main lang (%) (Top 10%)

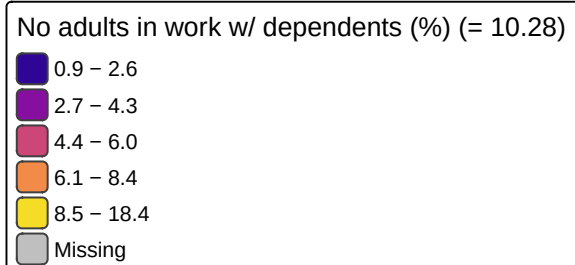
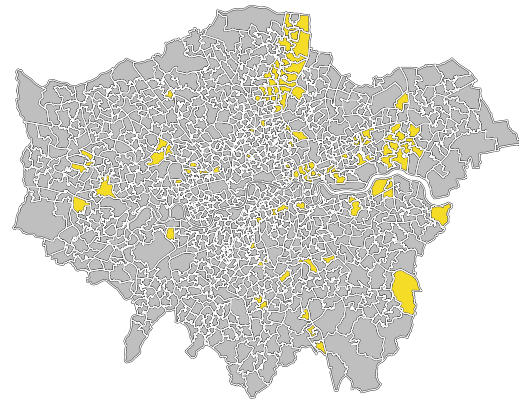


```
## [plot mode] fit legend/component: Some legend items or map components do not
## fit well, and are therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
## [plot mode] fit legend/component: Some legend items or map components do not
## fit well, and are therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
## Map saved to U:\Research project\figs\maps\combined_adults_in_employment_2011_census_percent_of_hous
##
## Resolution: 3000 by 1600 pixels
##
## Size: 13.63636 by 7.272727 inches (220 dpi)
```

No adults in work w/ dependents (%) in London



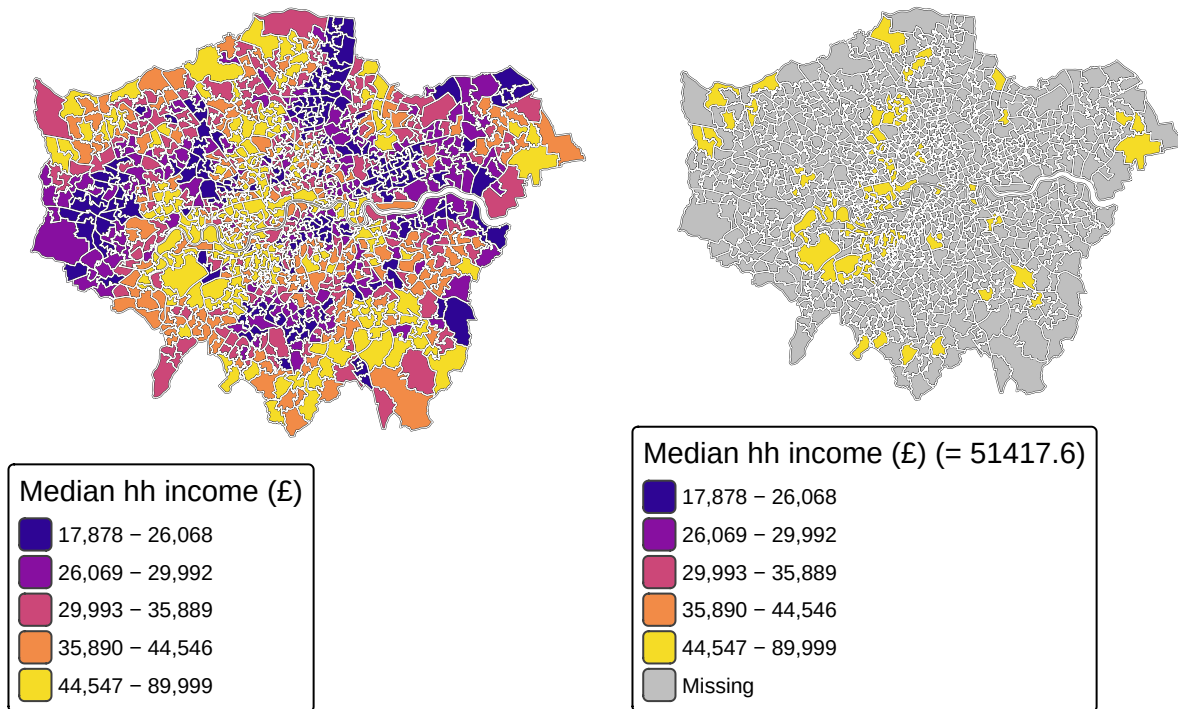
High areas: No adults in work w/ dependents (%) (Top 10%)



```
## [plot mode] fit legend/component: Some legend items or map components do not
## fit well, and are therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
## Map saved to U:\Research project\figs\maps\combined_household_income_estimates_2011_12_total_median_
##
## Resolution: 3000 by 1600 pixels
##
## Size: 13.63636 by 7.272727 inches (220 dpi)
```

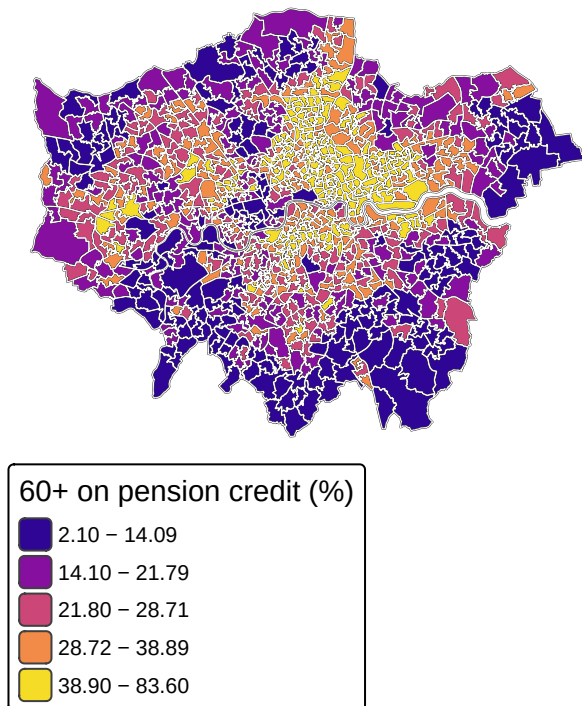
Median hh income (£) in London

High areas: Median hh income (£) (Top 10%)

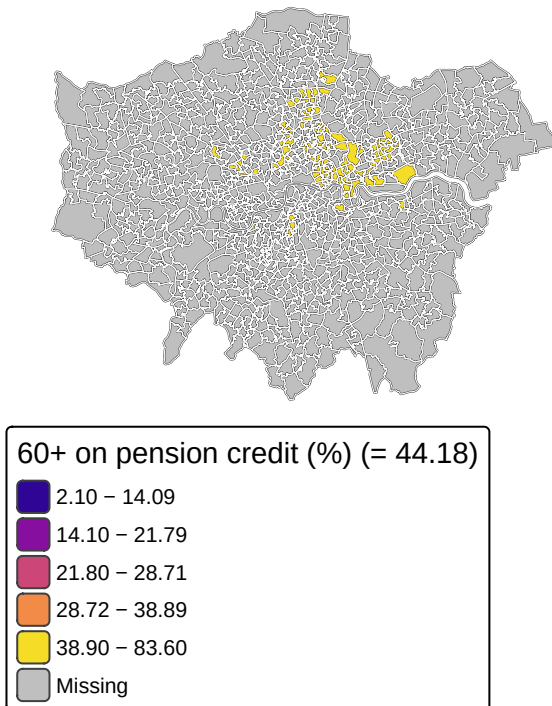


```
## [plot mode] fit legend/component: Some legend items or map components do not
## fit well, and are therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
## [plot mode] fit legend/component: Some legend items or map components do not
## fit well, and are therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
## Map saved to U:\Research project\figs\maps\combined_income_deprivation_2010_percent_of_people_aged_o
##
## Resolution: 3000 by 1600 pixels
##
## Size: 13.63636 by 7.272727 inches (220 dpi)
```

60+ on pension credit (%) in London

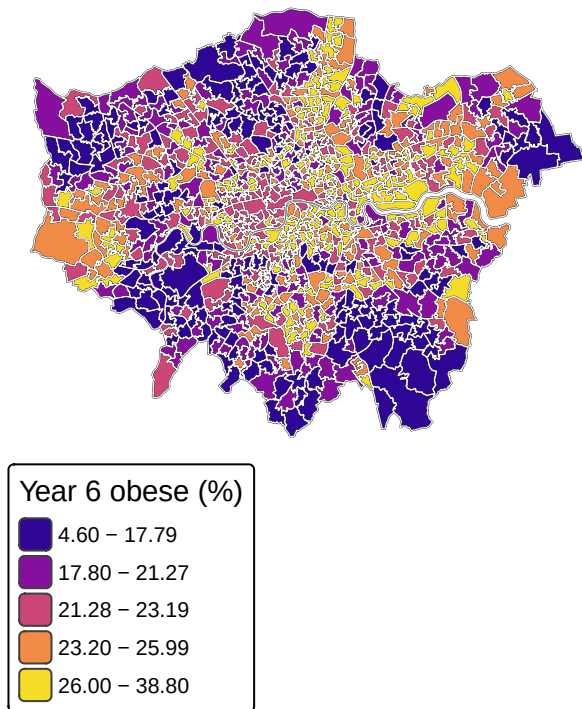


High areas: 60+ on pension credit (%) (Top 10%)

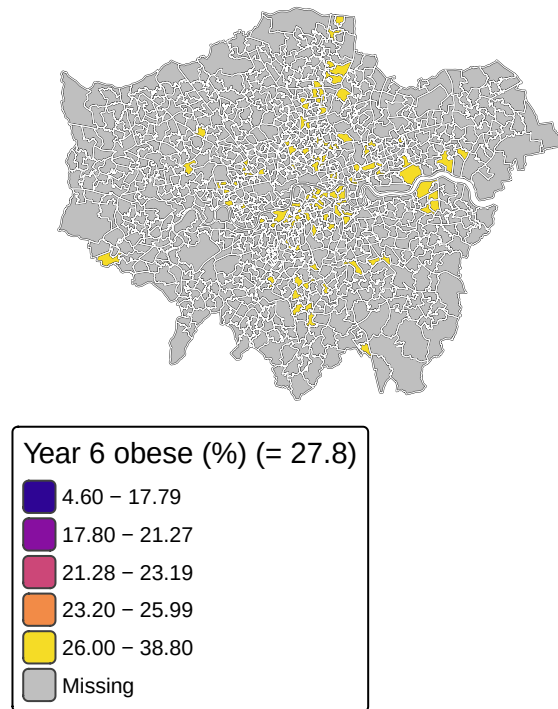


```
## [plot mode] fit legend/component: Some legend items or map components do not
## fit well, and are therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
## Map saved to U:\Research project\figs\maps\combined_obesity_percent_of_measured_children_in_year_6_w
##
## Resolution: 3000 by 1600 pixels
##
## Size: 13.63636 by 7.272727 inches (220 dpi)
```


Year 6 obese (%) in London



High areas: Year 6 obese (%) (Top 10%)



Checking the Model performance by Training and Testing Sets

```
set.seed(123) # for reproducibility

# Split data
n <- nrow(model_data_clean)
train_idx <- sample(seq_len(n), size = 0.7 * n)

train_data <- model_data_clean[train_idx, ]
test_data <- model_data_clean[-train_idx, ]

# Fitting model on training data
fit <- lm(formula(stepwise_model), data = train_data)

# Predicting on test data
test_data$predicted <- predict(fit, newdata = test_data)

# Calculating performance metrics
library(Metrics)
```

```
##
## Attaching package: 'Metrics'
```

```
## The following objects are masked from 'package:caret':
##
##   precision, recall

rmse_val <- rmse(test_data$energy_density, test_data$predicted)
mae_val  <- mae(test_data$energy_density, test_data$predicted)
r2_val   <- cor(test_data$energy_density, test_data$predicted)^2

cat("Test RMSE:", rmse_val, "\n")
```

```
## Test RMSE: 0.05717827
```

```
cat("Test MAE:", mae_val, "\n")
```

```
## Test MAE: 0.04515025
```

```
cat("Test R² :", r2_val, "\n")
```

```
## Test R² : 0.4532312
```

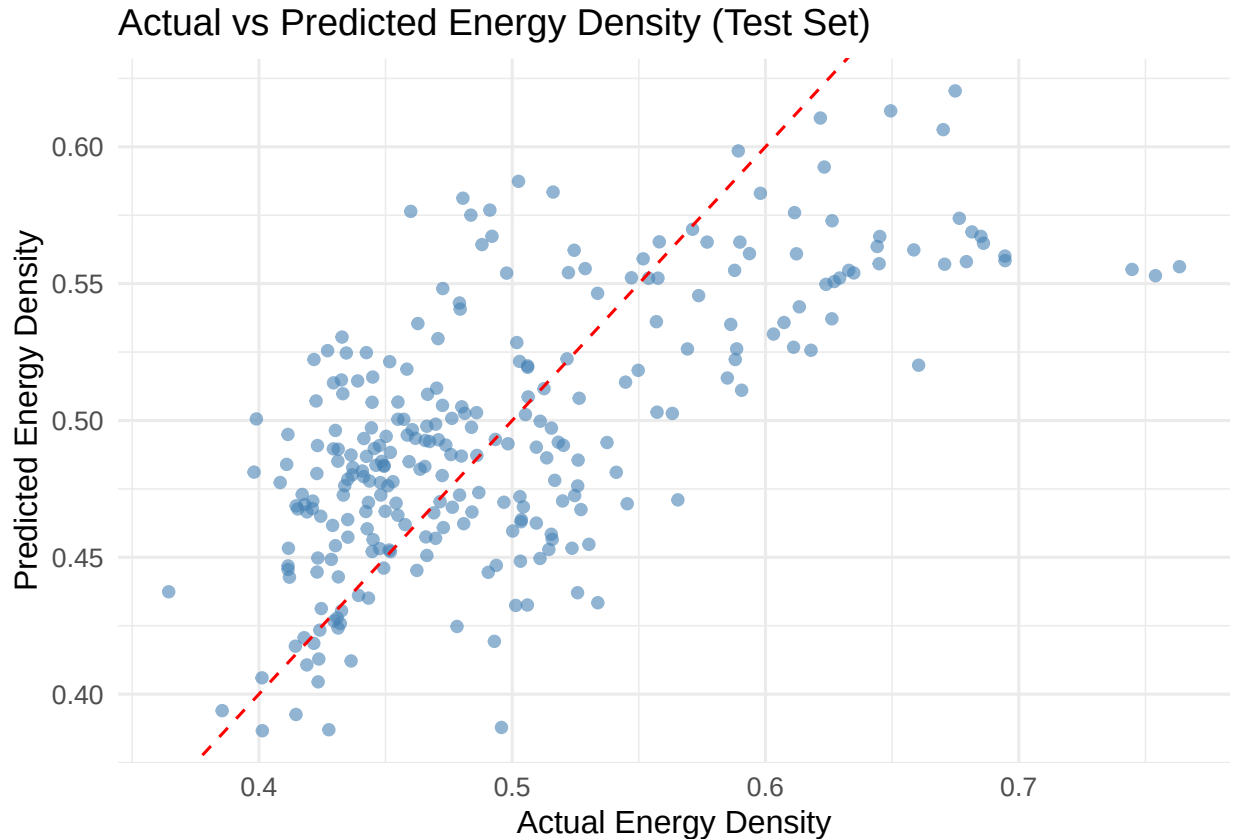
- The final stepwise regression model performed well, with an adjusted R square of 0.4405. This means that about 44% of the differences in energy density between regions can be explained by the sociodemographic factors included in the model. In other words, people living conditions — such as their income, household type, and local deprivation levels — help us understand why some areas tend to buy more calorie-dense foods than others.
- When the data was divided into training 70% and testing 30% sets, the model continued to perform well. It achieved a Test R square of 0.453, meaning it could explain about 45% of the variation in new, unseen data — almost the same as the training result. This consistency shows that the model is stable and not overfitted, which means it works well beyond the data it was trained on.
- The model prediction errors were small:

RMSE 0.057 and MAE 0.045 indicate that, on average, the model's predicted energy density values were very close to the actual observed values.

Overall, this suggests that variables such as household composition types of families, income levels, deprivation, language background, and obesity rates are strong indicators of regional differences in dietary health risks. Areas with lower incomes, higher deprivation, or more households without English as a main language tend to have higher average energy density, meaning their diets are generally less healthy.

visualising the Train and test sets

```
library(ggplot2)
p1 <- ggplot(test_data, aes(x = energy_density, y = predicted)) +
  geom_point(color = "steelblue", alpha = 0.6) +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +
  labs(title = "Actual vs Predicted Energy Density (Test Set)",
       x = "Actual Energy Density",
       y = "Predicted Energy Density") +
  theme_minimal(base_size = 12)
ggsave("figs/actual_vs_predicted_test.png", p1, width = 8, height = 6, dpi = 300)
print(p1)
```



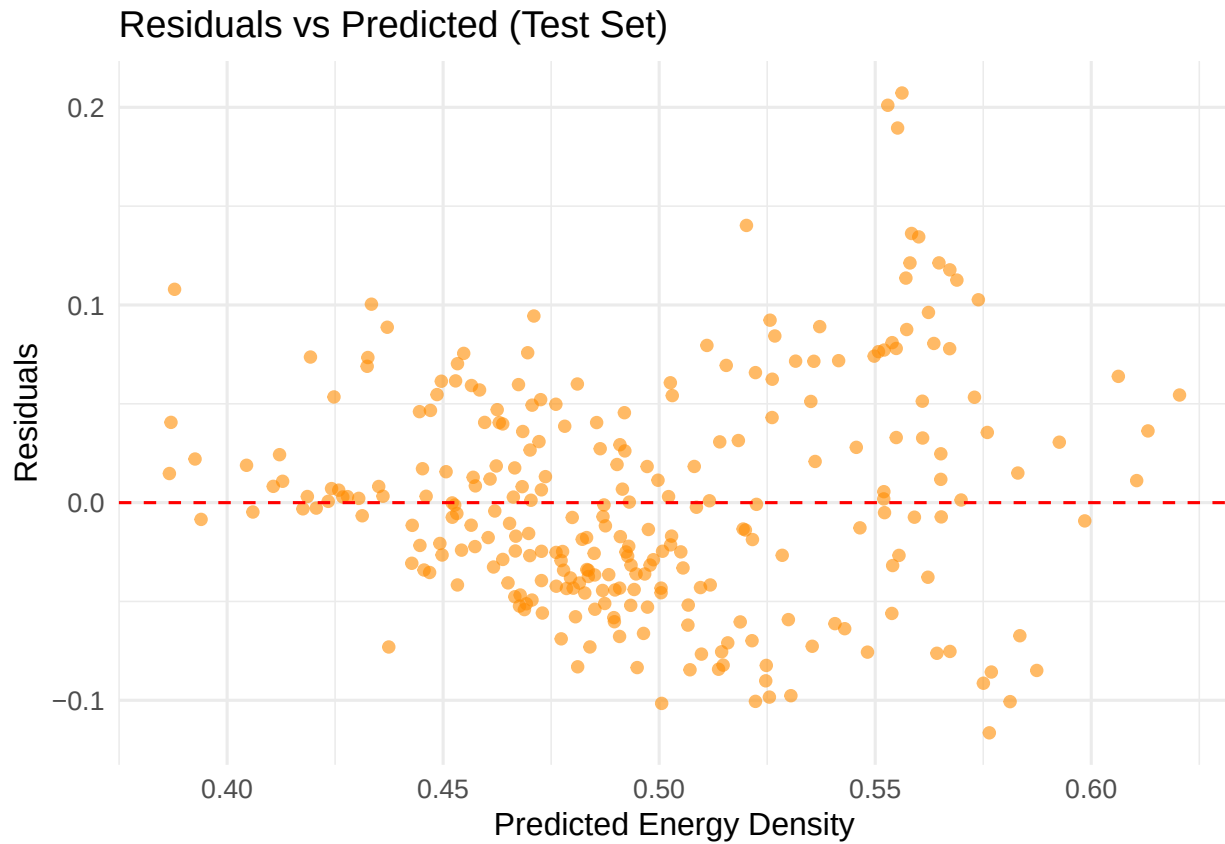
- This scatter plot shows how well the model's predicted energy density values match the actual values from the test data. Each blue dot represents one region in London.
- The x-axis shows the actual energy density observed in the data.
- The y-axis shows the predicted energy density from the model.
- The red dashed line represents perfect predictions if the model were 100% accurate, all points would fall exactly on this line.
- In this plot, most points are close to the red line, which means the model's predictions are quite accurate. There is some natural scatter, which is expected with real-world social data, but there are no extreme outliers. This shows that the model predicts energy density values consistently and without major bias (it doesn't systematically overpredict or underpredict).

Residual Plot

```
p2 <- ggplot(test_data, aes(x = predicted, y = energy_density - predicted)) +
  geom_point(alpha = 0.6, color = "darkorange") +
  geom_hline(yintercept = 0, color = "red", linetype = "dashed") +
  labs(title = "Residuals vs Predicted (Test Set)",
       x = "Predicted Energy Density",
       y = "Residuals") +
  theme_minimal(base_size = 12)
```



```
ggsave("figs/residuals_vs_predicted.png", p2, width = 8, height = 6, dpi = 300)
print(p2)
```



- This plot shows the residuals (the difference between actual and predicted energy density) against the predicted values from the model.
- The horizontal red dashed line represents zero error where the model's predictions are exactly correct.
- The orange points represent the prediction errors for each region.
- Most of the points are evenly scattered around the red line, with no clear upward or downward pattern.
- This means the model's errors are randomly distributed, which is a good sign. It shows that the model does not systematically overestimate or underestimate energy density in certain ranges.
- There are a few points slightly above or below the line, which is expected in real data, but overall the spread is narrow. This indicates that the model fits the data consistently and without major bias across all predicted values.

Comparing the Training and testing sets.

```
set.seed(123)

# Split data 70-30
```

```

n <- nrow(model_data_clean)
train_idx <- sample(seq_len(n), size = 0.7 * n)

train_data <- model_data_clean[train_idx, ]
test_data <- model_data_clean[-train_idx, ]

# Fit model on training data
fit <- lm(formula(stepwise_model), data = train_data)

# Predict on both training and test data
train_data$predicted <- predict(fit, newdata = train_data)
test_data$predicted <- predict(fit, newdata = test_data)

# Load Metrics package
library(Metrics)

# Compute metrics for both sets
train_rmse <- rmse(train_data$energy_density, train_data$predicted)
train_mae <- mae(train_data$energy_density, train_data$predicted)
train_r2 <- cor(train_data$energy_density, train_data$predicted)^2

test_rmse <- rmse(test_data$energy_density, test_data$predicted)
test_mae <- mae(test_data$energy_density, test_data$predicted)
test_r2 <- cor(test_data$energy_density, test_data$predicted)^2

# Combine metrics for comparison
comparison <- data.frame(
  Dataset = c("Training", "Testing"),
  RMSE = c(train_rmse, test_rmse),
  MAE = c(train_mae, test_mae),
  R2 = c(train_r2, test_r2)
)

print(comparison)

##      Dataset      RMSE      MAE      R2
## 1 Training 0.05692043 0.04374880 0.4407668
## 2 Testing 0.05717827 0.04515025 0.4532312

# -----
# Visual comparison: Actual vs Predicted plots
# -----
library(ggplot2)

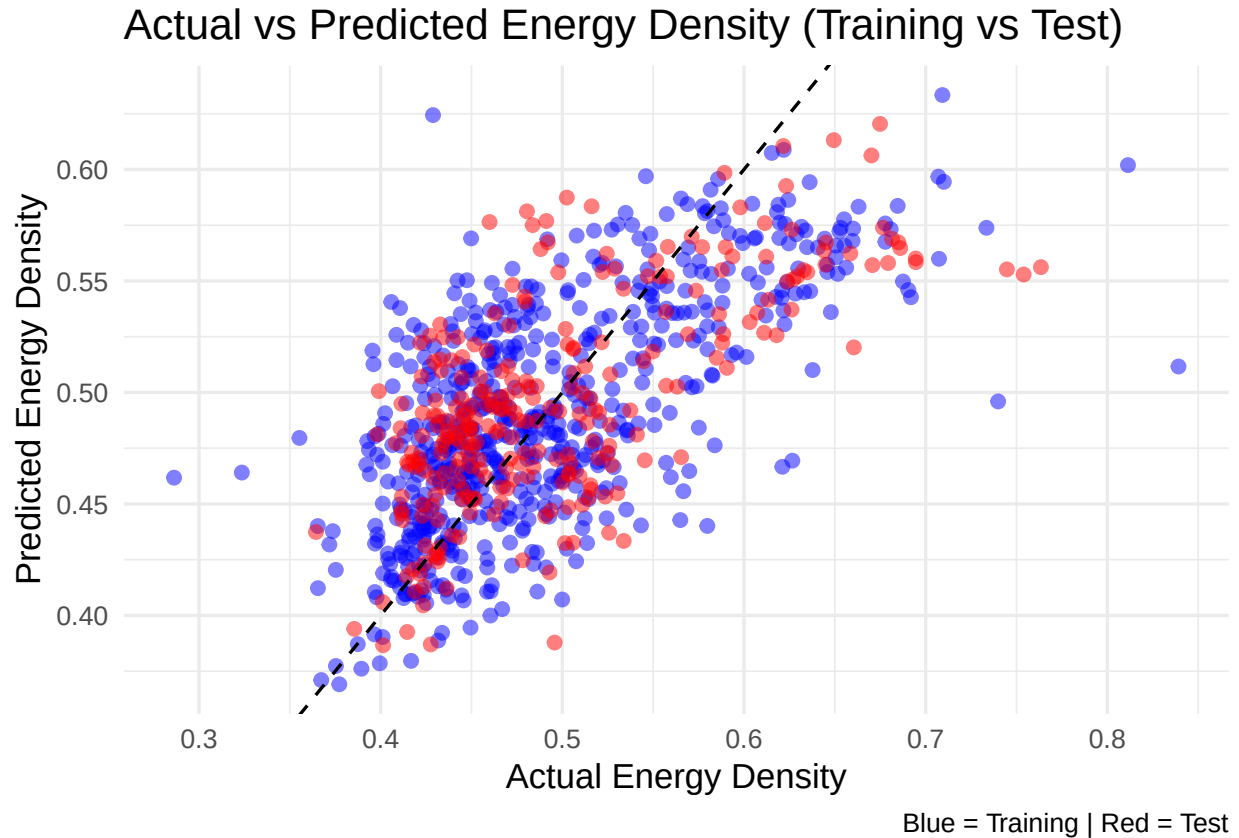
ggplot() +
  geom_point(data = train_data, aes(x = energy_density, y = predicted),
            color = "blue", alpha = 0.5, size = 2) +
  geom_point(data = test_data, aes(x = energy_density, y = predicted),
            color = "red", alpha = 0.5, size = 2) +
  geom_abline(intercept = 0, slope = 1, linetype = "dashed", color = "black") +
  labs(
    title = "Actual vs Predicted Energy Density (Training vs Test)",
    x = "Actual Energy Density",

```

```

y = "Predicted Energy Density",
caption = "Blue = Training | Red = Test"
) +
theme_minimal(base_size = 13)

```



```

ggsave(
  filename = "Final Figures/actual_vs_predicted_train_test.png",
  width = 10, height = 7, dpi = 300
)

```

- Figure X compares the actual and predicted energy density values for both the training (blue) and test (red) datasets. The clustering of points around the 1:1 dashed line indicates that the model predicts energy density accurately across regions.
- The close overlap between training and testing observations demonstrates consistent predictive behaviour across datasets, confirming that the stepwise regression model generalises well and does not exhibit overfitting.
- Minor deviations from the diagonal represent normal residual variation, but no systematic bias is evident, supporting the model's reliability and stability.

Comparing the Actual Energy Density and the Predicted Energy Density

```
# =====  
# Clear Actual vs Predicted Maps (no NAs, bright colours)  
# =====  
library(dplyr)  
library(sf)  
library(tmap)  
library(classInt)  
  
# --- 1) Prepare modelling dataset and predictions ---  
df_model <- stats_data %>%  
  dplyr::select(  
    msoa_code, energy_density,  
    household_composition_2011_percentages_couple_household_without_dependent_children,  
    household_composition_2011_percentages_lone_parent_household,  
    household_composition_2011_percentages_one_person_household,  
    household_language_2011_percent_of_households_where_no_people_in_household_have_english_as_a_main_language,  
    adults_in_employment_2011_census_percent_of_households_with_no_adults_in_employment_with_dependent_children,  
    household_income_estimates_2011_12_total_median_annual_household_income_upper_decile,  
    income_deprivation_2010_percent_of_people_aged_over_60_who_live_in_pension_credit_households,  
    obesity_percent_of_measured_children_in_year_6_who_were_classified_as_obese_2009_10_2011_12  
  ) %>%  
  tidyr::drop_na() # remove any rows with missing values  
  
# --- 2) Fit model (same formula as your stepwise one) ---  
fit_clean <- lm(formula(stepwise_model), data = df_model)  
  
# --- 3) Add predictions ---  
df_map <- df_model %>%  
  mutate(predicted = predict(fit_clean, newdata = df_model),  
         actual = energy_density) %>%  
  dplyr::select(msoa_code, actual, predicted)  
  
# --- 4) Join to spatial data ---  
map2 <- msoa_all %>%  
  left_join(df_map, by = "msoa_code") %>%  
  tidyr::drop_na(actual, predicted) # ensures only complete areas are shown  
  
# --- 5) Consistent colour breaks across both maps ---  
vals <- c(map2$actual, map2$predicted)  
brks <- classInt::classIntervals(vals, n = 5, style = "quantile")$brks  
  
# Bright, publication-friendly colour palette (alternatives: "viridis", "inferno", "roma")  
pal <- "batlow"  
  
tmap_mode("plot")  
  
## i tmap modes "plot" - "view"
```

```

map_actual <- tm_shape(map2) +
  tm_polygons(
    fill = "actual",
    fill.scale = tm_scale_intervals(
      breaks = brks, values = pal
    ),
    fill.legend = tm_legend(title = "Actual Energy Density (kcal/g)")
  ) +
  tm_borders("white", lwd = 0.4) +
  tm_title("Actual Energy Density (Observed)") +
  tm_layout(frame = FALSE, legend.outside = TRUE)

map_predicted <- tm_shape(map2) +
  tm_polygons(
    fill = "predicted",
    fill.scale = tm_scale_intervals(
      breaks = brks, values = pal
    ),
    fill.legend = tm_legend(title = "Predicted Energy Density (kcal/g)")
  ) +
  tm_borders("white", lwd = 0.4) +
  tm_title("Predicted Energy Density (Model Output)") +
  tm_layout(frame = FALSE, legend.outside = TRUE)

# --- 6) Display and save ---
panel <- tmap_arrange(map_actual, map_predicted, ncol = 2)
print(panel)

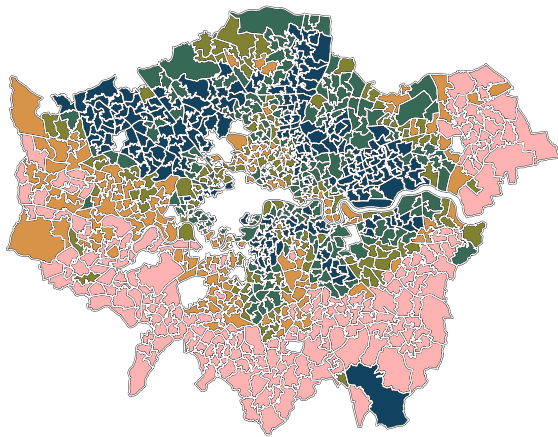
```

```

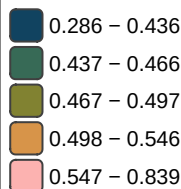
## [plot mode] fit legend/component: Some legend items or map components do not
## fit well, and are therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.

```

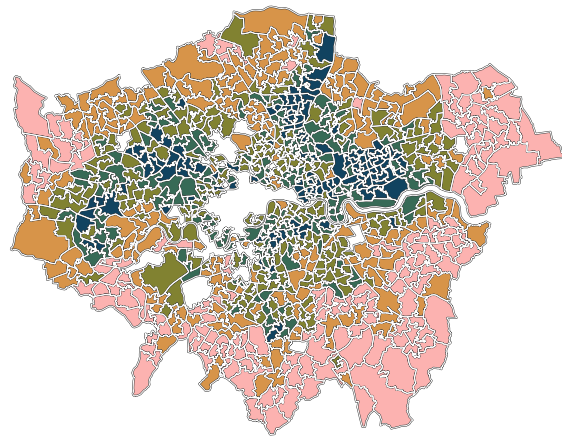
Actual Energy Density (Observed)



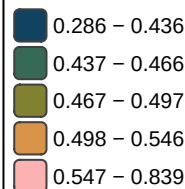
Actual Energy Density (kcal/g)



Predicted Energy Density (Model Output)



Predicted Energy Density (kcal/g)



```
if (!dir.exists("figs/maps")) dir.create("figs/maps", recursive = TRUE)
tmap_save(panel, "figs/maps/Actual_vs_Predicted_EnergyDensity_clean.png",
          width = 3200, height = 1500, dpi = 300)
```

```
## Map saved to U:\Research project\figs\maps\Actual_vs_Predicted_EnergyDensity_clean.png
## Resolution: 3200 by 1500 pixels
## Size: 10.66667 by 5 inches (300 dpi)
```

- To visualise how well the model captured regional variation in dietary energy density, both the observed (actual) and predicted values were mapped across London's MSOA regions. The analysis used the merged dataset that combined Tesco grocery data (providing energy density) with MSOA-level sociodemographic indicators from the London Atlas. From this dataset, only the variables identified as significant in the stepwise regression were retained, and all rows with missing values were removed to ensure accuracy.
- Using the final regression model, predicted energy density values were generated for each MSOA region based on its demographic and socioeconomic characteristics. These predictions were then linked back to the corresponding geographic boundaries in the MSOA shapefile to enable spatial visualisation.
- Both the actual and predicted energy density maps were created using the tmap package in R. Identical colour scales and quantile based classification breaks were applied to both maps to ensure direct visual comparability. Areas with higher predicted or actual energy density values are shown in darker shades, highlighting regions where diets are likely to be more energy dense. By comparing the two maps side by side, it is possible to assess how closely the model's predictions align with the real spatial distribution of dietary energy density across London.